

CSES Graph Algorithms

A Problem-Solving Companion to the CSES Problem Set

Maruri Saikiran

August 11, 2026

Contents

1	Graph Traversal	1
1.1	BFS and DFS: One Idea, Two Orders	1
1.2	Counting Rooms	2
1.3	Labyrinth	3
1.4	Message Route	5
1.5	Building Roads	7
1.6	Building Teams	8
1.7	Round Trip	10
1.8	Monsters	12
2	Shortest Paths	15
2.1	Relaxation, Three Ways	15
2.2	Shortest Routes I	16
2.3	Shortest Routes II	17
2.4	High Score	19
2.5	Flight Discount	20
2.6	Cycle Finding	22
2.7	Flight Routes	23
3	DAGs, Topological Order, and Strong Connectivity	27
3.1	Order and Components in Directed Graphs	27
3.2	Round Trip II	28
3.3	Course Schedule	29
3.4	Longest Flight Route	31
3.5	Game Routes	33
3.6	Investigation	34
3.7	Planets and Kingdoms	36
3.8	Flight Routes Check	38
3.9	Giant Pizza	39
3.10	Coin Collector	42
4	Successor and Functional Graphs	45
4.1	One Outgoing Edge Changes Everything	45
4.2	Planets Queries I	46
4.3	Planets Queries II	47
4.4	Planets Cycles	49
5	Minimum Spanning Trees	53
5.1	Greedy Connection with Disjoint Set	53
5.2	Road Reparation	54
5.3	Road Construction	55
6	Eulerian and Hamiltonian Paths	59
6.1	Every Edge Once vs Every Node Once	59
6.2	Mail Delivery	60
6.3	De Bruijn Sequence	61

6.4	Teleporters Path	63
6.5	Hamiltonian Flights	65
6.6	Knight's Tour	66
7	Maximum Flow and Matching	69
7.1	Everything Is a Flow	69
7.2	Download Speed	70
7.3	Police Chase	72
7.4	School Dance	74
7.5	Distinct Routes	76

Chapter 1

Graph Traversal

Every graph problem starts with a traversal. BFS and DFS are the same idea – visit a node, then visit its unvisited neighbours – differing only in whether a queue or a stack (recursion) decides who is next. BFS’s level-by-level order gives shortest paths in unweighted graphs; DFS’s dive-deep order exposes components, cycles, and structure. This chapter is the foundation the rest of the book stands on.

1.1 BFS and DFS: One Idea, Two Orders

A traversal marks nodes visited so none is processed twice, and explores outward from a source. The only choice is the order neighbours are taken.

Queue for Shortest, Stack for Structure

BFS uses a queue: it visits all nodes at distance 1, then 2, and so on, so the first time it reaches a node is via a shortest (fewest-edges) path. **DFS** uses recursion (an implicit stack): it follows one path as deep as possible before backtracking, which naturally reveals connected components, cycles (a back edge to an ancestor), and bipartiteness (a 2-colouring conflict).

What Each Problem Here Needs

- **Count/flood connected regions** – DFS or BFS flood fill: Counting Rooms.
- **Shortest path / reconstruct path in an unweighted graph or grid** – BFS: Labyrinth, Message Route.
- **Is everything connected / how many edges to connect** – component counting: Building Roads.
- **2-colour the graph** – BFS/DFS bipartite check: Building Teams.
- **Find a cycle** – DFS back edge (or DSU) in an undirected graph: Round Trip.
- **Multi-source spread / escape** – BFS from many sources at once: Monsters.

Each section gives the CSES-style input handling, the traversal, path or answer reconstruction where required, a dry run, complete C++, and complexity.

1.2 Counting Rooms

Problem Statement. Given an $n \times m$ grid of floor (.) and wall (#) cells, count the number of rooms: maximal groups of floor cells connected through shared edges (four directions).

Pattern Recognition

“Count connected regions in a grid” is a **flood fill**: whenever an unvisited floor cell is found, start a traversal that marks its entire room, and increment a counter. Each new traversal is exactly one room.

Flood fill counts components

Scan every cell. When you hit a floor cell not yet visited, it belongs to a new room, so increment the room count and run a BFS or DFS that marks all floor cells reachable from it (moving only up/down/left/right through floor). Because each cell is marked the first time it is reached and never revisited, the number of traversals launched equals the number of rooms.

One Launch per Component

The visited grid guarantees each cell is consumed once. So the count of “fresh starts” – unvisited floor cells that trigger a new flood – is the component count. This is the grid form of connected-component counting.

Algorithm

1. For each cell (i, j) : if it is floor and unvisited, increment the room count and flood-fill from it.
2. The flood marks every floor cell reachable through the four directions.
3. Output the room count.

Dry run

Dry Run. 3×3 grid: row0 “.#.”, row1 “.#.”, row2 “...”.

- Start at $(0, 0)$: flood the left column and bottom row – one room.
- $(0, 2)$ connects down to $(1, 2)$ then to the bottom row, already visited – so it is the *same* room via the bottom.
- Here everything floor is one connected room, so the answer is 1.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, m;
5 vector<string> g;
6 vector<vector<bool>> vis;
7
8 void bfs(int si, int sj) {
9     queue<pair<int, int>> q;
10    q.push({si, sj}); vis[si][sj] = true;

```

```

11  int dr[] = {-1,1,0,0}, dc[] = {0,0,-1,1};
12  while (!q.empty()) {
13      auto [r, c] = q.front(); q.pop();
14      for (int k = 0; k < 4; k++) {
15          int nr = r + dr[k], nc = c + dc[k];
16          if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
17          if (g[nr][nc] == '#' || vis[nr][nc]) continue;
18          vis[nr][nc] = true;
19          q.push({nr, nc});
20      }
21  }
22 }
23
24 int main() {
25     cin >> n >> m;
26     g.resize(n);
27     for (auto& row : g) cin >> row;
28     vis.assign(n, vector<bool>(m, false));
29
30     int rooms = 0;
31     for (int i = 0; i < n; i++)
32         for (int j = 0; j < m; j++)
33             if (g[i][j] == '.' && !vis[i][j]) { rooms++; bfs(i, j); }
34
35     cout << rooms << "\n";
36     return 0;
37 }

```

Complexity Analysis

Time Complexity. $O(nm)$ – each cell is visited once. **Space Complexity.** $O(nm)$ for the visited grid and BFS queue.

Key Takeaway

Counting rooms is connected-component counting on an implicit grid graph: one flood fill per unvisited region. This is the simplest graph traversal and the template for every grid problem that follows.

1.3 Labyrinth

Problem Statement. Given a grid with a start A, a target B, floor ., and walls #, find a shortest path from A to B moving in the four directions. Output YES, its length, and the sequence of moves (U/D/L/R), or NO.

Pattern Recognition

A grid where each move costs one step is an unweighted graph, so **BFS** finds the shortest path, and remembering *how* each cell was first reached (which move and from where) reconstructs the actual path.

BFS plus per-cell parent

Run BFS from A. The first time BFS reaches a cell, it does so by a shortest path, so record the move (U/D/L/R) used to enter it and its predecessor. When B is reached, walk the parent

pointers back to A, collecting the moves, then reverse them. If BFS finishes without reaching B, output NO.

Store the Move That First Reached Each Cell

Distances alone say how long the path is; storing the direction taken into each cell lets you replay the path. Because BFS's first arrival is optimal, these stored moves spell out one shortest route.

Algorithm

1. BFS from A; for each newly visited cell store the move character that entered it.
2. On reaching B, backtrack via parents, collect moves, reverse.
3. Output YES, the length, and the move string; else NO.

Dry run

Dry Run. A at (0,0), B at (0,2), top row open.

- BFS reaches (0,1) via R, then (0,2) via R.
- Backtrack from B: moves R,R; reversed gives RR, length 2.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n, m;
6      cin >> n >> m;
7      vector<string> g(n);
8      for (auto& row : g) cin >> row;
9
10     int sr, sc, er, ec;
11     for (int i = 0; i < n; i++)
12         for (int j = 0; j < m; j++) {
13             if (g[i][j] == 'A') { sr = i; sc = j; }
14             if (g[i][j] == 'B') { er = i; ec = j; }
15         }
16
17     vector<vector<char>> move(n, vector<char>(m, 0));
18     vector<vector<bool>> vis(n, vector<bool>(m, false));
19     int dr[] = {-1,1,0,0}, dc[] = {0,0,-1,1};
20     char mv[] = {'U','D','L','R'};
21
22     queue<pair<int,int>> q;
23     q.push({sr, sc}); vis[sr][sc] = true;
24     while (!q.empty()) {
25         auto [r, c] = q.front(); q.pop();
26         for (int k = 0; k < 4; k++) {
27             int nr = r + dr[k], nc = c + dc[k];
28             if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
29             if (g[nr][nc] == '#' || vis[nr][nc]) continue;
30             vis[nr][nc] = true;

```

```

31         move[nr][nc] = mv[k];
32         q.push({nr, nc});
33     }
34 }
35
36 if (!vis[er][ec]) { cout << "NO\n"; return 0; }
37 string path;
38 int r = er, c = ec;
39 while (!(r == sr && c == sc)) {
40     char mvc = move[r][c];
41     path += mvc;
42     if (mvc == 'U') r++; else if (mvc == 'D') r--;
43     else if (mvc == 'L') c++; else c--; // step back
44 }
45 reverse(path.begin(), path.end());
46 cout << "YES\n" << path.size() << "\n" << path << "\n";
47 return 0;
48 }

```

Complexity Analysis

Time Complexity. $O(nm)$ – one BFS over the grid. **Space Complexity.** $O(nm)$ for visited and move grids.

Key Takeaway

Shortest path on an unweighted grid is BFS; storing the entering move per cell turns the distance into a printable route. The backtrack-and-reverse pattern reappears in every “print the path” problem, including Message Route next.

1.4 Message Route

Problem Statement. Given an undirected graph of n computers and m connections, find a route with the fewest edges from computer 1 to computer n . Output the number of computers on the route and the route itself, or IMPOSSIBLE.

Pattern Recognition

Fewest edges in an unweighted graph is **BFS**. Recording each node’s predecessor lets the shortest route be reconstructed by walking parents back from n to 1.

BFS with parent pointers

BFS from node 1. The first time a node is dequeued-into, it is via a shortest route, so store the node it came from. When n is reached, follow *parent* from n back to 1, then reverse to get the route in forward order. If n is never visited, the answer is IMPOSSIBLE.

Parents Turn Distances into Routes

This is Labyrinth on a general graph rather than a grid: the same BFS, the same parent-backtrack. On any unweighted graph, “fewest edges” plus “print the path” is always BFS plus predecessors.

Algorithm

1. Build adjacency lists. BFS from 1, storing $\text{parent}[v]$ on first visit.
2. If n unvisited, print IMPOSSIBLE.
3. Else backtrack $n \rightarrow \dots \rightarrow 1$, reverse, and print the count and route.

Dry run

Dry Run. Edges 1–2, 2–3, 1–3; $n = 3$.

- BFS from 1 reaches 3 directly (edge 1–3), parent $[3] = 1$.
- Route $1 \rightarrow 3$: two computers. (The two-edge path 1–2–3 is longer and not chosen.)

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;
7     vector<vector<int>> adj(n + 1);
8     for (int i = 0; i < m; i++) {
9         int a, b; cin >> a >> b;
10        adj[a].push_back(b);
11        adj[b].push_back(a);
12    }
13
14    vector<int> parent(n + 1, 0);
15    vector<bool> vis(n + 1, false);
16    queue<int> q;
17    q.push(1); vis[1] = true;
18    while (!q.empty()) {
19        int u = q.front(); q.pop();
20        for (int v : adj[u])
21            if (!vis[v]) { vis[v] = true; parent[v] = u; q.push(v); }
22    }
23
24    if (!vis[n]) { cout << "IMPOSSIBLE\n"; return 0; }
25    vector<int> route;
26    for (int v = n; v != 0; v = parent[v]) route.push_back(v);
27    reverse(route.begin(), route.end());
28
29    cout << route.size() << "\n";
30    for (int v : route) cout << v << " ";
31    cout << "\n";
32    return 0;
33 }
```

Complexity Analysis

Time Complexity. $O(n + m)$ – one BFS. **Space Complexity.** $O(n + m)$ for adjacency lists, parents, and the queue.

Key Takeaway

Message Route is the graph (not grid) version of shortest-path-with-route: BFS for fewest edges, parent pointers for the path. Master this pair – BFS distance and parent reconstruction – as the reflex for all unweighted shortest-path queries.

1.5 Building Roads

Problem Statement. Given n cities and m existing roads, find the minimum number of new roads needed so that every city is reachable from every other, and output which roads to build.

Pattern Recognition

“Connect all components with the fewest new edges” reduces to counting connected components: if there are c components, exactly $c - 1$ new roads suffice – one linking a representative of each component to a common one.

Components, then bridge them

Find the connected components (by DFS/BFS or Disjoint Set). Pick one representative city per component. Building a road from the first component’s representative to each other representative connects everything with $c - 1$ roads, which is optimal – no fewer can connect c separate groups. Collect and print those representative pairs.

 c Components Need Exactly $c - 1$ Links

Each new road can reduce the number of components by at most one, so going from c components to 1 requires at least $c - 1$ roads; connecting one representative per component to a fixed anchor achieves that bound.

Algorithm

1. Find components; record one representative node per component.
2. Output (count $- 1$) as the number of roads.
3. For each representative after the first, print a road from the first representative to it.

Dry run

Dry Run. $n = 4$, roads 1–2; cities 3 and 4 isolated.

- Components: $\{1, 2\}$, $\{3\}$, $\{4\}$ – reps 1, 3, 4.
- Build 1–3 and 1–4: two roads. Answer 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;

```

```

7   vector<vector<int>> adj(n + 1);
8   for (int i = 0; i < m; i++) {
9       int a, b; cin >> a >> b;
10      adj[a].push_back(b);
11      adj[b].push_back(a);
12  }
13
14  vector<bool> vis(n + 1, false);
15  vector<int> reps;
16  for (int s = 1; s <= n; s++) {
17      if (vis[s]) continue;
18      reps.push_back(s);           // one representative
19      // per component
20      queue<int> q; q.push(s); vis[s] = true;
21      while (!q.empty()) {
22          int u = q.front(); q.pop();
23          for (int v : adj[u]) if (!vis[v]) { vis[v] = true; q.push(v); }
24      }
25
26      cout << reps.size() - 1 << "\n";
27      for (size_t i = 1; i < reps.size(); i++)
28          cout << reps[0] << " " << reps[i] << "\n";
29      return 0;
30  }

```

Complexity Analysis

Time Complexity. $O(n + m)$ – one traversal to find components. **Space Complexity.** $O(n + m)$.

Key Takeaway

Minimum edges to connect a graph is (components $- 1$); connect one representative per component to a common anchor. Recognising “make it all connected” as component counting – solvable by traversal or Disjoint Set – is the reusable idea.

1.6 Building Teams

Problem Statement. Given n players and m friendships, assign each player to one of two teams so that no two friends are on the same team. Output a valid assignment, or IMPOSSIBLE.

Pattern Recognition

“Split into two groups so every edge crosses” is **2-colouring**, i.e. testing bipartiteness. BFS or DFS colours each node opposite its neighbour; a conflict (an edge between same-coloured nodes) means an odd cycle exists and the answer is IMPOSSIBLE.

Two-colour via traversal

Colour a start node 1, its neighbours 2, their neighbours 1, and so on, alternating on every edge. If the traversal ever tries to colour a node the same as an already-coloured neighbour, the graph contains an odd cycle and cannot be 2-coloured. Repeat over all components, since the graph may be disconnected.

Bipartite iff No Odd Cycle

Alternating colours along edges is consistent exactly when every cycle has even length. The first same-colour edge encountered is a certificate of an odd cycle, so the traversal both constructs the teams and detects impossibility.

Algorithm

1. For each uncoloured node, BFS/DFS colouring neighbours with the opposite colour.
2. If a neighbour already has the same colour, output IMPOSSIBLE.
3. Otherwise output every node's colour (1 or 2).

Dry run

Dry Run. Friendships 1–2, 2–3; $n = 3$.

- Colour 1=1, 2=2, 3=1. No same-colour edge.
- Teams: player 1 and 3 on team 1, player 2 on team 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;
7     vector<vector<int>> adj(n + 1);
8     for (int i = 0; i < m; i++) {
9         int a, b; cin >> a >> b;
10        adj[a].push_back(b);
11        adj[b].push_back(a);
12    }
13
14    vector<int> color(n + 1, 0);
15    for (int s = 1; s <= n; s++) {
16        if (color[s]) continue;
17        color[s] = 1;
18        queue<int> q; q.push(s);
19        while (!q.empty()) {
20            int u = q.front(); q.pop();
21            for (int v : adj[u]) {
22                if (!color[v]) { color[v] = 3 - color[u]; q.push(v); }
23                else if (color[v] == color[u]) { cout << "IMPOSSIBLE\n"
24                    ; return 0; }
25            }
26        }
27    }
28    for (int i = 1; i <= n; i++) cout << color[i] << " ";
29    cout << "\n";
30    return 0;
31 }
```

Complexity Analysis

Time Complexity. $O(n + m)$ – one traversal. **Space Complexity.** $O(n + m)$.

Key Takeaway

Two-team assignment is bipartite 2-colouring: alternate colours along edges, and a same-colour edge proves an odd cycle. The trick 3 – color flips between 1 and 2 cleanly. This underlies conflict-graph and scheduling feasibility checks.

1.7 Round Trip

Problem Statement. Given an undirected graph of n nodes and m edges, find any cycle (a route that starts and ends at the same node using distinct edges) and print the sequence of nodes, or IMPOSSIBLE.

Pattern Recognition

“Find a cycle in an undirected graph” is a **DFS** that watches for a **back edge**: an edge to an already-visited node that is not the immediate parent. Such an edge closes a cycle, which is reconstructed from the DFS parent chain.

Back edge closes a cycle

Run DFS tracking each node’s parent. If DFS at node u finds a neighbour v that is already visited and v is not u ’s parent, then v is an ancestor of u and the tree path from v down to u , plus the edge $u \rightarrow v$, forms a cycle. Reconstruct it by following parents from u up to v .

Not-the-Parent Is the Whole Condition

In an undirected graph every edge appears from both ends, so seeing the parent again is not a cycle. A visited neighbour that is *not* the parent is a genuine back edge – the signal of a cycle. Recording parents lets you print it.

Algorithm

1. DFS from each unvisited node, storing parents.
2. On finding a visited non-parent neighbour v from u : walk parents $u \rightarrow \dots \rightarrow v$, append v again to close the loop, and print.
3. If no back edge is ever found, print IMPOSSIBLE.

Dry run

Dry Run. Edges 1–2, 2–3, 3–1.

- DFS $1 \rightarrow 2 \rightarrow 3$; from 3, neighbour 1 is visited and not 3’s parent (2) – back edge.
- Cycle: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, m;
5  vector<vector<int>> adj;
6  vector<int> parent;
7  vector<bool> vis;
8  int cycleStart = -1, cycleEnd = -1;
9
10 bool dfs(int u, int p) {
11     vis[u] = true;
12     parent[u] = p;
13     for (int v : adj[u]) {
14         if (v == p) continue;           // skip the edge back
15         // to parent
16         if (vis[v]) { cycleStart = v; cycleEnd = u; return true; }
17         if (dfs(v, u)) return true;
18     }
19     return false;
20 }
21
22 int main() {
23     cin >> n >> m;
24     adj.assign(n + 1, {});
25     for (int i = 0; i < m; i++) {
26         int a, b; cin >> a >> b;
27         adj[a].push_back(b);
28         adj[b].push_back(a);
29     }
30     parent.assign(n + 1, 0);
31     vis.assign(n + 1, false);
32
33     for (int s = 1; s <= n; s++)
34         if (!vis[s] && dfs(s, -1)) break;
35
36     if (cycleStart == -1) { cout << "IMPOSSIBLE\n"; return 0; }
37     vector<int> cyc;
38     for (int v = cycleEnd; v != cycleStart; v = parent[v]) cyc.
39         push_back(v);
40     cyc.push_back(cycleStart);
41     cyc.push_back(cycleEnd);           // close the loop
42     reverse(cyc.begin(), cyc.end());
43
44     cout << cyc.size() << "\n";
45     for (int v : cyc) cout << v << " ";
46     cout << "\n";
47     return 0;
48 }

```

Complexity Analysis

Time Complexity. $O(n + m)$ – one DFS. **Space Complexity.** $O(n + m)$, plus recursion depth $O(n)$.

Key Takeaway

Undirected cycle detection is a DFS back edge to a non-parent ancestor; parents reconstruct the cycle. (Disjoint Set also detects a cycle – the first edge joining two already-connected nodes – but does not print it as directly.)

1.8 Monsters

Problem Statement. On a grid with your start **A**, monsters **M**, walls **#**, and floor **.**, you and all monsters move one step per turn (four directions). Escape by reaching any boundary cell before a monster occupies it. Output **YES** and a path, or **NO**.

Pattern Recognition

This is a race modelled with **multi-source BFS**. First compute, for every cell, the earliest time a monster can arrive (BFS from all monsters at once). Then BFS from **A**, only allowed to enter a cell *strictly before* any monster could – reaching the boundary means escape.

Two BFS passes: monsters first, then you

Run a BFS seeded with *all* monster cells simultaneously; this labels each cell with the minimum monster arrival time. Then BFS from **A** with your own time counter, stepping into a neighbour only if your arrival time there is strictly less than the monster time (so a monster cannot be there yet). Track parents to print the path. If your BFS reaches any boundary cell, you escape; otherwise output **NO**.

Multi-Source BFS Gives Simultaneous Spread

Seeding a BFS queue with every monster at once yields, in one pass, the soonest a monster reaches each cell – as if all monsters spread in parallel. Comparing your arrival time against that field decides which cells are safe to enter.

Algorithm

1. Multi-source BFS from all **M** cells to get *monsterTime*[].
2. BFS from **A** tracking *myTime* and parents; enter a cell only if *myTime* < *monsterTime*.
3. If a boundary cell is reached, reconstruct and print the path (**YES**); else **NO**.

Dry run

Dry Run. **A** near the centre with one distant **M**.

- Monster BFS marks cells near **M** with small times, cells near **A** with large times.
- Your BFS heads to the nearest boundary, reaching it before the monster's time there – escape with a printed path.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;
7     vector<string> g(n);
8     for (auto& row : g) cin >> row;
9
10    const int INF = 1e9;
11    vector<vector<int>> mt(n, vector<int>(m, INF)); // monster
12    // arrival time
13    queue<pair<int,int>> q;
14    int sr = 0, sc = 0;
15    for (int i = 0; i < n; i++)
16        for (int j = 0; j < m; j++) {
17            if (g[i][j] == 'M') { mt[i][j] = 0; q.push({i, j}); }
18            if (g[i][j] == 'A') { sr = i; sc = j; }
19        }
20    int dr[] = {-1,1,0,0}, dc[] = {0,0,-1,1};
21    char mvc[] = {'U','D','L','R'};
22
23    while (!q.empty()) { // multi-source
24        BFS
25        auto [r, c] = q.front(); q.pop();
26        for (int k = 0; k < 4; k++) {
27            int nr = r + dr[k], nc = c + dc[k];
28            if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
29            if (g[nr][nc] == '#' || mt[nr][nc] != INF) continue;
30            mt[nr][nc] = mt[r][c] + 1;
31            q.push({nr, nc});
32        }
33    }
34
35    vector<vector<int>> myt(n, vector<int>(m, INF));
36    vector<vector<char>> mv(n, vector<char>(m, 0));
37    myt[sr][sc] = 0;
38    q.push({sr, sc});
39    int er = -1, ec = -1;
40    while (!q.empty()) {
41        auto [r, c] = q.front(); q.pop();
42        if (r == 0 || r == n - 1 || c == 0 || c == m - 1) { er = r; ec
43            = c; break; }
44        for (int k = 0; k < 4; k++) {
45            int nr = r + dr[k], nc = c + dc[k];
46            if (nr < 0 || nr >= n || nc < 0 || nc >= m) continue;
47            if (g[nr][nc] == '#' || myt[nr][nc] != INF) continue;
48            if (myt[r][c] + 1 >= mt[nr][nc]) continue; // monster
49            // reaches first
50            myt[nr][nc] = myt[r][c] + 1;
51            mv[nr][nc] = mvc[k];
52            q.push({nr, nc});
53        }
54    }
55
56    if (er == -1) { cout << "NO\n"; return 0; }
57    string path;

```

```
54     int r = er, c = ec;
55     while (!(r == sr && c == sc)) {
56         char ch = mv[r][c]; path += ch;
57         if (ch == 'U') r++; else if (ch == 'D') r--;
58         else if (ch == 'L') c++; else c--;
59     }
60     reverse(path.begin(), path.end());
61     cout << "YES\n" << path.size() << "\n" << path << "\n";
62     return 0;
63 }
```

Complexity Analysis

Time Complexity. $O(nm)$ – two BFS passes over the grid. **Space Complexity.** $O(nm)$ for the time and move grids.

Key Takeaway

Races against spreading hazards are multi-source BFS (the hazard field) plus a guarded single-source BFS (your moves), entering a cell only when you strictly beat the hazard. This two-field comparison is the template for fire/flood-escape problems.

Chapter 2

Shortest Paths

Weighted shortest paths are all the same operation – relaxation – applied in different orders. Dijkstra relaxes from the nearest node using a heap; Bellman-Ford relaxes every edge repeatedly and survives negative weights; Floyd-Warshall relaxes through every intermediate node for all pairs at once. The CSES shortest-path problems tour these three engines and their twists.

2.1 Relaxation, Three Ways

Relaxing an edge $u \rightarrow v$ of weight w means: if $dist[u] + w < dist[v]$, improve $dist[v]$. Which edges, and in what order, distinguishes the algorithms.

Choosing the Shortest-Path Engine

- **Single source, non-negative weights – Dijkstra** with a priority queue: Shortest Routes I, and (with a twist) Flight Discount and Flight Routes.
- **All pairs, small n – Floyd-Warshall**: Shortest Routes II.
- **Negative weights / detect a negative cycle – Bellman-Ford**: High Score, Cycle Finding.
- **k -th shortest / few best paths** – Dijkstra keeping the best k arrival costs per node: Flight Routes.
- **One special edge (a discount) – a layered** (state $\times$ used-discount) Dijkstra: Flight Discount.

Layering Encodes Extra State in the Node

Several twists (used a discount yet? how many stops so far?) are handled by running the same algorithm on an expanded graph whose nodes are (original node, extra state). The algorithm does not change; only the node set grows.

2.2 Shortest Routes I

Problem Statement. Given a directed graph of n cities and m roads with non-negative lengths, output the shortest distance from city 1 to every city.

Pattern Recognition

Single source, non-negative weights: **Dijkstra's algorithm**. Always finalise the unvisited node with the smallest tentative distance (found via a min-heap), then relax its outgoing edges.

Greedy finalisation with a heap

Keep $dist[1] = 0$ and all others ∞ . Repeatedly pop the smallest (d, u) from a priority queue; if it is stale (a better distance is already recorded) skip it, else relax each edge $u \rightarrow v$ of weight w : if $dist[u] + w < dist[v]$, update and push v . Non-negativity guarantees a popped node's distance is final.

Non-Negativity Makes the Nearest Node Final

When the closest unfinished node is popped, no later path can beat its distance, because any such path would pass through a node already at least as far and then add a non-negative edge. This is why Dijkstra is correct only without negative edges.

Algorithm

1. $dist[1] = 0$; push $(0, 1)$.
2. Pop smallest; skip if stale; else relax all out-edges, pushing improved neighbours.
3. Output $dist[1 \dots n]$.

Dry run

Dry Run. Edges $1 \rightarrow 2(5)$, $1 \rightarrow 3(2)$, $3 \rightarrow 2(1)$.

- Pop $(0, 1)$: relax $2 \rightarrow 5$, $3 \rightarrow 2$. Pop $(2, 3)$: relax 2 via 3: $2 + 1 = 3 < 5$.
- Distances: $dist = [0, 3, 2]$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;
7     vector<vector<pair<int, long long>>> adj(n + 1);    // {to, weight}
8     for (int i = 0; i < m; i++) {
9         int a, b; long long w; cin >> a >> b >> w;
10        adj[a].push_back({b, w});
11    }
12
13    vector<long long> dist(n + 1, LLONG_MAX);
14    dist[1] = 0;

```

```

15 priority_queue<pair<long long,int>, vector<pair<long long,int>>,
16         greater<>> pq;
17 pq.push({0, 1});
18 while (!pq.empty()) {
19     auto [d, u] = pq.top(); pq.pop();
20     if (d > dist[u]) continue;           // stale
21     for (auto& [v, w] : adj[u])
22         if (dist[u] + w < dist[v]) {
23             dist[v] = dist[u] + w;
24             pq.push({dist[v], v});
25         }
26 }
27 for (int i = 1; i <= n; i++) cout << dist[i] << " ";
28 cout << "\n";
29 return 0;
30 }

```

Complexity Analysis

Time Complexity. $O((n + m) \log n)$. **Space Complexity.** $O(n + m)$.

Key Takeaway

Shortest Routes I is textbook Dijkstra: heap of tentative distances, skip stale pops, relax outgoing edges. It is the single-source backbone the Flight Discount and Flight Routes twists build on.

2.3 Shortest Routes II

Problem Statement. Given an undirected weighted graph of n cities and m roads, answer q queries, each asking the shortest distance between two cities. Output -1 if unreachable. Here n is small (up to a few hundred).

Pattern Recognition

Many shortest-distance queries between arbitrary pairs, with small n , is **Floyd–Warshall**: compute all-pairs shortest distances once in $O(n^3)$, then answer each query in $O(1)$.

Relax through every intermediate

Let $dist[i][j]$ start as the direct edge (or ∞ , and 0 on the diagonal). For each candidate intermediate node k , update every pair: $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$. After considering all k , every entry is the true shortest distance, because any shortest path's highest-indexed intermediate is handled in the round for that k .

Allow One More Intermediate at a Time

The k -loop is outermost: after iteration k , $dist[i][j]$ is the shortest path using intermediates only from $\{1, \dots, k\}$. Expanding the allowed set one node at a time builds all-pairs distances without missing any path.

Algorithm

1. Initialise the distance matrix from edges (keep the minimum for multi-edges; 0 on the diagonal).

2. Triple loop k, i, j relaxing through k (guard against ∞ overflow).
3. Answer each query from the matrix; -1 if still ∞ .

Dry run

Dry Run. Triangle 1–2(4), 2–3(1), 1–3(10).

- Through $k = 2$: $\text{dist}[1][3] = \min(10, 4 + 1) = 5$.
- Query $(1, 3) \Rightarrow 5$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const long long INF = 1e18;
4
5 int main() {
6     int n, m, q;
7     cin >> n >> m >> q;
8     vector<vector<long long>> dist(n + 1, vector<long long>(n + 1, INF)
9         );
10    for (int i = 1; i <= n; i++) dist[i][i] = 0;
11    for (int i = 0; i < m; i++) {
12        int a, b; long long w; cin >> a >> b >> w;
13        dist[a][b] = min(dist[a][b], w);
14        dist[b][a] = min(dist[b][a], w);
15    }
16
17    for (int k = 1; k <= n; k++)
18        for (int i = 1; i <= n; i++)
19            for (int j = 1; j <= n; j++)
20                if (dist[i][k] < INF && dist[k][j] < INF)
21                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
22
23    while (q--) {
24        int a, b; cin >> a >> b;
25        cout << (dist[a][b] >= INF ? -1 : dist[a][b]) << "\n";
26    }
27    return 0;
28 }
```

Complexity Analysis

Time Complexity. $O(n^3 + q)$. **Space Complexity.** $O(n^2)$ for the distance matrix.

Key Takeaway

All-pairs shortest distance with small n and many queries is Floyd–Warshall: one $O(n^3)$ triple loop, $O(1)$ queries. The k -outermost order – allowing one more intermediate each round – is the detail that makes it correct.

2.4 High Score

Problem Statement. A directed graph of n rooms and m edges, each edge carrying a score (possibly negative). Starting in room 1 and ending in room n , maximise the total collected score. If the score can be made arbitrarily large, output -1 .

Pattern Recognition

Maximising with possibly negative (and positive) edges, plus detecting an unbounded answer, is **Bellman–Ford** run for maxima. A positive cycle that lies on some $1 \rightarrow n$ path makes the score unbounded – the analogue of a negative cycle in the minimisation form.

Maximise, then check reachable positive cycles

Set $dist[1] = 0$, others $-\infty$, and relax every edge $n-1$ times using max: $dist[v] = \max(dist[v], dist[u] + w)$. After that, run one more round: any node whose value still increases sits on or downstream of a positive cycle. Mark such nodes and propagate the mark forward; if node n ends up marked and is reachable from 1, the answer is -1 . Otherwise $dist[n]$ is the maximum score.

A Positive Cycle on the Path Means Unbounded

Only cycles that are both reachable from 1 and can reach n matter – looping such a cycle adds score forever. Extra relaxation rounds detect the still-improving nodes, and a forward propagation checks whether n is affected.

Algorithm

1. Relax all edges $n - 1$ times with max (from $dist[1] = 0$).
2. Run n more rounds; if an edge still improves v , mark $v = +\infty$ and propagate the mark along edges.
3. If $dist[n] = +\infty$ output -1 , else output $dist[n]$.

Dry run

Dry Run. $1 \rightarrow 2(5)$, $2 \rightarrow 3(-2)$, $3 \rightarrow 2(3)$ (cycle $2 \rightarrow 3 \rightarrow 2$ gains $+1$), $3 \rightarrow 4(1)$; $n = 4$.

- Cycle $2 \rightarrow 3 \rightarrow 2$ has net $+1$ and reaches 4 – score is unbounded.
- Output -1 .

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const long long NEG = LLONG_MIN / 4;
4
5 int main() {
6     int n, m;
7     cin >> n >> m;
8     vector<array<long long, 3>> edges(m);           // {u, v, w}
9     for (auto& e : edges) cin >> e[0] >> e[1] >> e[2];
10
11     vector<long long> dist(n + 1, NEG);

```

```

12     dist[1] = 0;
13     for (int i = 0; i < n - 1; i++)
14         for (auto& [u, v, w] : edges)
15             if (dist[u] > NEG) dist[v] = max(dist[v], dist[u] + w);
16
17     // extra rounds: mark nodes on/after a positive cycle as +infinity
18     for (int i = 0; i < n; i++)
19         for (auto& [u, v, w] : edges)
20             if (dist[u] > NEG && dist[u] + w > dist[v]) dist[v] =
21                 LLONG_MAX / 4;
22
23     cout << (dist[n] >= LLONG_MAX / 8 ? -1 : dist[n]) << "\n";
24     return 0;
25 }

```

Complexity Analysis

Time Complexity. $O(nm)$ – Bellman–Ford’s relaxation rounds. **Space Complexity.** $O(n + m)$.

Key Takeaway

Maximising a path score with mixed-sign edges is Bellman–Ford with max; a reachable positive cycle makes it unbounded. Running extra rounds and propagating a “ $+\infty$ ” mark cleanly detects the nodes an unbounded cycle can inflate – the mirror of negative-cycle detection.

2.5 Flight Discount

Problem Statement. A directed graph of n cities and m flights with prices. You hold one discount coupon that halves the price of a single flight. Find the minimum total price to travel from city 1 to city n .

Pattern Recognition

“One special action, used at most once” is a **layered Dijkstra**: double the state to (city, coupon used?). The algorithm is unchanged; only the graph grows to two copies, with discounted edges crossing from the “unused” layer to the “used” layer.

Two copies of the graph

Run Dijkstra over states $(u, 0)$ (coupon still available) and $(u, 1)$ (coupon spent). From $(u, 0)$, a normal edge $u \rightarrow v$ of price w leads to $(v, 0)$ at cost w , or you spend the coupon to reach $(v, 1)$ at cost $w/2$. From $(u, 1)$, only normal edges to $(v, 1)$ are allowed. The answer is $\text{dist}[(n, 1)]$ – reaching n having used the coupon (using it on the cheapest beneficial edge is handled automatically by taking the minimum).

Encode “Coupon Used” in the Node

Rather than special-casing which edge to discount, expand the state space so the coupon status is part of the node. Dijkstra then explores all possibilities and the shortest path to $(n, 1)$ is optimal by construction.

Algorithm

1. States $(u, 0)$ and $(u, 1)$; start $(1, 0)$ at cost 0.
2. Relax: $(u, 0) \rightarrow (v, 0)$ cost w ; $(u, 0) \rightarrow (v, 1)$ cost $w/2$; $(u, 1) \rightarrow (v, 1)$ cost w .
3. Answer $\text{dist}[(n, 1)]$.

Dry run

Dry Run. $1 \rightarrow 2(10)$, $2 \rightarrow 3(6)$; $n = 3$.

- Discount the pricier edge $1 \rightarrow 2$: $5 + 6 = 11$. Discount $2 \rightarrow 3$: $10 + 3 = 13$.
- Minimum is 11.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;
7     vector<vector<pair<int, long long>>> adj(n + 1);
8     for (int i = 0; i < m; i++) {
9         int a, b; long long w; cin >> a >> b >> w;
10        adj[a].push_back({b, w});
11    }
12
13    // dist[u][0/1] : 0 = coupon unused, 1 = coupon used
14    vector<array<long long, 2>> dist(n + 1, {LLONG_MAX, LLONG_MAX});
15    dist[1][0] = 0;
16    priority_queue<tuple<long long, int, int>,
17                  vector<tuple<long long, int, int>>, greater<>> pq;
18    pq.push({0, 1, 0});
19    while (!pq.empty()) {
20        auto [d, u, used] = pq.top(); pq.pop();
21        if (d > dist[u][used]) continue;
22        for (auto& [v, w] : adj[u]) {
23            if (d + w < dist[v][used]) { // normal edge,
24                // same layer
25                dist[v][used] = d + w; pq.push({dist[v][used], v, used});
26            }
27            if (used == 0 && d + w / 2 < dist[v][1]) { // spend coupon
28                // here
29                dist[v][1] = d + w / 2; pq.push({dist[v][1], v, 1});
30            }
31        }
32    }
33    cout << dist[n][1] << "\n";
34    return 0;
35 }

```

Complexity Analysis

Time Complexity. $O((n + m) \log n)$ – Dijkstra on a graph of double size. **Space Complexity.** $O(n + m)$.

Key Takeaway

A single one-time special move becomes an extra boolean in the state, and plain Dijkstra on the layered graph solves it. This state-augmentation trick generalises to “at most k specials” (a $k + 1$ -layer graph) and is one of the most reusable shortest-path patterns.

2.6 Cycle Finding

Problem Statement. Given a directed graph of n nodes and m edges with (possibly negative) weights, find a negative cycle and print the nodes on it, or output **NO CYCLE**.

Pattern Recognition

Detecting a negative cycle is the failure mode of **Bellman–Ford**: if any edge can still be relaxed after $n - 1$ rounds, a negative cycle is reachable. Following predecessor pointers back n steps lands on the cycle, which is then unwound.

Relax n times, then trace back

Initialise all distances to 0 (so every node is a potential start) and relax all edges n times, recording a predecessor on each successful relaxation. If some edge relaxes on the n -th round, its endpoint x is affected by a negative cycle. Stepping from x back through predecessors n times guarantees landing on a node y that is truly on the cycle; following predecessors from y until y repeats reconstructs the cycle.

The n -th Relaxation Exposes a Negative Cycle

Shortest paths use at most $n - 1$ edges, so a further improvement can only come from a cycle with negative total weight. Walking back n predecessors from the improved node skips the “tail” into the cycle, ensuring the printed sequence is the cycle itself.

Algorithm

1. $dist[i] = 0$ for all i ; relax all edges n times, storing predecessors; remember the last node improved on round n .
2. If none improved, print **NO CYCLE**.
3. Else step back n times to reach a cycle node, then collect the cycle by following predecessors until it repeats; print it.

Dry run

Dry Run. $1 \rightarrow 2(1)$, $2 \rightarrow 3(-3)$, $3 \rightarrow 1(1)$ (cycle sum -1).

- After repeated relaxation, an edge keeps improving – a negative cycle exists.
- Tracing back yields the cycle $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;
7     vector<array<long long,3>> edges(m);
8     for (auto& e : edges) cin >> e[0] >> e[1] >> e[2];
9
10    vector<long long> dist(n + 1, 0);
11    vector<int> pre(n + 1, -1);
12    int x = -1;
13    for (int i = 0; i < n; i++) {
14        x = -1;
15        for (auto& [u, v, w] : edges)
16            if (dist[u] + w < dist[v]) {
17                dist[v] = dist[u] + w;
18                pre[v] = u;
19                x = v;                // node improved this
20                round
21            }
22    }
23
24    if (x == -1) { cout << "NO CYCLE\n"; return 0; }
25    for (int i = 0; i < n; i++) x = pre[x];    // step into the
26    cycle
27
28    vector<int> cyc;
29    for (int v = x; ; v = pre[v]) {
30        cyc.push_back(v);
31        if (v == x && cyc.size() > 1) break;
32    }
33    reverse(cyc.begin(), cyc.end());
34    cout << "YES\n";
35    for (int v : cyc) cout << v << " ";
36    cout << "\n";
37    return 0;
38 }

```

Complexity Analysis

Time Complexity. $O(nm)$ – Bellman–Ford relaxation. **Space Complexity.** $O(n + m)$.

Key Takeaway

Negative-cycle finding is Bellman–Ford relaxed one extra round, then a back-walk of n predecessors to land firmly on the cycle. Initialising all distances to 0 makes every node a virtual source, so any negative cycle anywhere is caught.

2.7 Flight Routes

Problem Statement. Given a directed graph of n cities and m flights with prices, find the k cheapest route prices from city 1 to city n . Routes may reuse cities and flights, and different routes with equal price are listed separately.

Pattern Recognition

The k cheapest arrival costs is a **modified Dijkstra** in which each node may be finalised up to k times. Pop costs in increasing order; the first k times node n is popped are its k cheapest route prices.

Let each node pop k times

Standard Dijkstra finalises each node once. Here, keep a counter of how many times each node has been popped from the min-heap. Pop the smallest cost; if the popped node has already been finalised k times, skip it; otherwise increment its counter and, only while the counter is $\leq k$, relax its edges (pushing $cost + w$ for each). The k costs recorded when node n is popped, in order, are the answer.

Bounded Multiplicity Gives k-Shortest

Because the heap always yields the globally smallest unfinalised cost, the i -th time a node is popped corresponds to its i -th cheapest arrival. Capping each node at k pops keeps the heap small while still producing the k best costs to the destination.

Algorithm

1. Min-heap seeded with $(0, 1)$; a pop-count array.
2. Pop smallest (d, u) ; if u popped k times, skip; else count it, record if $u = n$, and relax edges.
3. Output the k recorded costs for node n .

Dry run

Dry Run. $1 \rightarrow 2(1)$, $1 \rightarrow 2(3)$, $2 \rightarrow 3(1)$; $k = 2$, $n = 3$.

- Cheapest to 3: $1+1 = 2$; second cheapest: $3+1 = 4$.
- Output 2 4.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m, k;
6     cin >> n >> m >> k;
7     vector<vector<pair<int, long long>>> adj(n + 1);
8     for (int i = 0; i < m; i++) {
9         int a, b; long long w; cin >> a >> b >> w;
10        adj[a].push_back({b, w});
11    }
12
13    vector<int> cnt(n + 1, 0);
14    vector<long long> best;
15    priority_queue<pair<long long, int>, vector<pair<long long, int>>,
16                  greater<>> pq;
17    pq.push({0, 1});

```

```

18     while (!pq.empty()) {
19         auto [d, u] = pq.top(); pq.pop();
20         if (cnt[u] >= k) continue;           // already have k
21         for u
22         cnt[u]++;
23         if (u == n) best.push_back(d);
24         if (cnt[u] > k) continue;
25         for (auto& [v, w] : adj[u])
26             if (cnt[v] < k) pq.push({d + w, v});
27     }
28     for (long long c : best) cout << c << " ";
29     cout << "\n";
30     return 0;
}

```

Complexity Analysis

Time Complexity. $O(mk \log(mk))$ – each edge may be pushed up to k times. **Space Complexity.** $O(mk)$ for the heap in the worst case.

Key Takeaway

The k shortest (repeatable) route costs come from a Dijkstra that lets each node be finalised k times. Capping pops per node bounds the work while the heap's increasing-cost order guarantees the first k arrivals at the sink are the cheapest.

Chapter 3

DAGs, Topological Order, and Strong Connectivity

Direction changes everything. A directed acyclic graph has a topological order that turns path counting and longest paths into simple DP. A general directed graph may have cycles, but contracting each strongly connected component to a point recovers a DAG – and that condensation, plus its cousin 2-SAT, solves a surprising range of problems.

3.1 Order and Components in Directed Graphs

Two structural facts drive this chapter: a DAG can be linearised (topological sort), and any directed graph condenses into a DAG of its strongly connected components (SCCs).

Directed-Graph Toolkit

- **Detect a directed cycle / order tasks – topological sort** (DFS colours or Kahn’s queue): Round Trip II, Course Schedule.
- **Longest path / count paths in a DAG – DP in topological order**: Longest Flight Route, Game Routes, Investigation.
- **Group mutually reachable nodes – SCC** (Kosaraju or Tarjan): Planets and Kingdoms, Flight Routes Check.
- **Condense to a DAG, then DP** – SCC condensation: Coin Collector.
- **Satisfy boolean constraints of the form “ a or b ” – 2-SAT** via the implication graph’s SCCs: Giant Pizza.

Condensation Turns Any Directed Graph into a DAG

Contract each SCC into a single super-node. The result is always acyclic, so topological-order DP applies to it. This “find SCCs, condense, then DP on the DAG” pipeline is the backbone of the hard problems here.

3.2 Round Trip II

Problem Statement. Given a *directed* graph of n nodes and m edges, find a directed cycle and print its nodes in order, or IMPOSSIBLE.

Pattern Recognition

A directed cycle is found by **DFS with three colours**: white (unvisited), grey (on the current recursion stack), black (fully processed). An edge to a *grey* node is a back edge – it closes a directed cycle.

Grey means “on the current path”

Colour a node grey when DFS enters it and black when it finishes. During DFS at u , if a neighbour v is grey, then v is an ancestor still on the recursion stack, so the path $v \rightarrow \dots \rightarrow u$ plus the edge $u \rightarrow v$ is a directed cycle. Reconstruct it via parent pointers from u back to v . A neighbour that is black is already finished and off the path, so it does not indicate a cycle (unlike the undirected case, direction matters).

Back Edge to a Grey Ancestor

The colour of the target distinguishes a cycle-closing back edge (grey, still on the stack) from a harmless cross/forward edge (black, done). Tracking parents lets the grey-to-current path be printed as the cycle.

Algorithm

1. DFS from each white node; mark grey on entry, black on exit, storing parents.
2. On finding a grey neighbour v from u : record v (cycle start) and u (cycle end), stop.
3. Reconstruct $v \rightarrow \dots \rightarrow u \rightarrow v$ and print; else IMPOSSIBLE.

Dry run

Dry Run. $1 \rightarrow 2, 2 \rightarrow 3, 3 \rightarrow 1$.

- DFS greys 1, 2, 3; from 3 the edge to grey 1 is a back edge.
- Cycle $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, m;
5 vector<vector<int>> adj;
6 vector<int> color, parent;           // 0 white, 1 grey, 2
   black
7 int cs = -1, ce = -1;
8
9 bool dfs(int u) {
10     color[u] = 1;
11     for (int v : adj[u]) {
12         if (color[v] == 0) {

```

```

13     parent[v] = u;
14     if (dfs(v)) return true;
15 } else if (color[v] == 1) {           // back edge to grey
16     ancestor
17     cs = v; ce = u; return true;
18 }
19 color[u] = 2;
20 return false;
21 }
22
23 int main() {
24     cin >> n >> m;
25     adj.assign(n + 1, {});
26     for (int i = 0; i < m; i++) {
27         int a, b; cin >> a >> b;
28         adj[a].push_back(b);
29     }
30     color.assign(n + 1, 0);
31     parent.assign(n + 1, 0);
32
33     for (int s = 1; s <= n; s++)
34         if (color[s] == 0 && dfs(s)) break;
35
36     if (cs == -1) { cout << "IMPOSSIBLE\n"; return 0; }
37     vector<int> cyc;
38     cyc.push_back(cs);
39     for (int v = ce; v != cs; v = parent[v]) cyc.push_back(v);
40     cyc.push_back(cs);
41     reverse(cyc.begin(), cyc.end());
42     cout << cyc.size() << "\n";
43     for (int v : cyc) cout << v << " ";
44     cout << "\n";
45     return 0;
46 }

```

Complexity Analysis

Time Complexity. $O(n+m)$ – one DFS. **Space Complexity.** $O(n+m)$, recursion depth $O(n)$ (use an explicit stack for very large n).

Key Takeaway

Directed cycle detection is DFS with grey/black colours: a back edge points to a grey (on-stack) ancestor. The colour distinction – absent in the undirected version – is exactly what respects edge direction.

3.3 Course Schedule

Problem Statement. Given n courses and m prerequisite relations (a directed edge $a \rightarrow b$ means a must be taken before b), output a valid order to complete all courses, or **IMPOSSIBLE** if the prerequisites are cyclic.

Pattern Recognition

Ordering tasks under “before” constraints is **topological sort**. Kahn’s algorithm repeatedly outputs a node with no remaining prerequisites (in-degree 0); if some nodes never reach in-degree 0, a cycle exists and no order is possible.

Kahn’s in-degree queue

Compute each node’s in-degree (number of prerequisites). Put all in-degree- 0 nodes in a queue. Repeatedly pop one, append it to the order, and decrement the in-degree of its successors; any successor reaching 0 joins the queue. If the produced order contains all n nodes, it is a valid topological order; if fewer, the leftover nodes form a cycle and the answer is **IMPOSSIBLE**.

In-Degree Zero Means “Ready Now”

A node can be scheduled once all its prerequisites are done – i.e. its in-degree has dropped to 0. Draining nodes in this readiness order both produces a valid schedule and detects a cycle (some nodes never become ready).

Algorithm

1. Build the graph and in-degrees; queue all in-degree-0 nodes.
2. Pop a node, output it, decrement successors’ in-degrees, enqueue new zeros.
3. If all n nodes were output, print the order; else **IMPOSSIBLE**.

Dry run

Dry Run. Edges $1 \rightarrow 2$, $1 \rightarrow 3$, $3 \rightarrow 2$.

- In-degrees: 1:0, 2:2, 3:1. Start with 1; then 3 becomes 0; then 2.
- Order 1, 3, 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;
7     vector<vector<int>>> adj(n + 1);
8     vector<int> indeg(n + 1, 0);
9     for (int i = 0; i < m; i++) {
10         int a, b; cin >> a >> b;
11         adj[a].push_back(b);
12         indeg[b]++;
13     }
14
15     queue<int> q;
16     for (int i = 1; i <= n; i++) if (indeg[i] == 0) q.push(i);
17
18     vector<int> order;
19     while (!q.empty()) {

```

```

20     int u = q.front(); q.pop();
21     order.push_back(u);
22     for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
23 }
24
25 if ((int)order.size() != n) { cout << "IMPOSSIBLE\n"; return 0; }
26 for (int v : order) cout << v << " ";
27 cout << "\n";
28 return 0;
29 }

```

Complexity Analysis

Time Complexity. $O(n + m)$. **Space Complexity.** $O(n + m)$.

Key Takeaway

Scheduling under precedence is topological sort; Kahn's in-degree queue both produces the order and detects cyclic (hence infeasible) constraints when fewer than n nodes emerge. This is the standard dependency-resolution algorithm (build systems, task runners).

3.4 Longest Flight Route

Problem Statement. Given a *directed acyclic* graph of n cities and m flights, find a route from city 1 to city n visiting the maximum number of cities, and print it, or IMPOSSIBLE.

Pattern Recognition

Longest path in a **DAG** is a topological-order DP – unlike general graphs (where longest path is NP-hard), acyclicity makes it linear. Process nodes so that every predecessor is finalised first, and each node takes the best incoming route.

DP over topological order

Let $dp[v]$ be the maximum number of cities on a route from 1 to v . Initialise $dp[1] = 1$ and others $-\infty$ (unreachable). Processing nodes in topological order, relax each edge $u \rightarrow v$: if $dp[u] + 1 > dp[v]$, set $dp[v] = dp[u] + 1$ and record u as v 's predecessor. Because u precedes v topologically, $dp[u]$ is final when used. Reconstruct the route by following predecessors back from n .

Acyclicity Turns Longest Path into DP

In a DAG the topological order guarantees each node's best value is computed before it is needed downstream, so a single sweep finds the longest path. The predecessor array turns the length into a printable route.

Algorithm

1. Topologically sort (Kahn's). $dp[1] = 1$, rest $-\infty$.
2. In topological order, relax out-edges, updating dp and predecessors.
3. If $dp[n] < 0$ print IMPOSSIBLE; else backtrack from n and print the route.

Dry run

Dry Run. $1 \rightarrow 2, 1 \rightarrow 3, 2 \rightarrow 3, 3 \rightarrow 4; n = 4$.

- *dp*: 1:1, 2:2, 3:3 (via $1 \rightarrow 2 \rightarrow 3$), 4:4.
- Longest route $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$, four cities.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;
7     vector<vector<int>> adj(n + 1);
8     vector<int> indeg(n + 1, 0);
9     for (int i = 0; i < m; i++) {
10         int a, b; cin >> a >> b;
11         adj[a].push_back(b);
12         indeg[b]++;
13     }
14
15     // topological order via Kahn's
16     vector<int> order, deg = indeg;
17     queue<int> q;
18     for (int i = 1; i <= n; i++) if (deg[i] == 0) q.push(i);
19     while (!q.empty()) {
20         int u = q.front(); q.pop(); order.push_back(u);
21         for (int v : adj[u]) if (--deg[v] == 0) q.push(v);
22     }
23
24     const int NEG = -1e9;
25     vector<int> dp(n + 1, NEG), pre(n + 1, 0);
26     dp[1] = 1;
27     for (int u : order)
28         if (dp[u] > NEG)
29             for (int v : adj[u])
30                 if (dp[u] + 1 > dp[v]) { dp[v] = dp[u] + 1; pre[v] = u; }
31
32     if (dp[n] < 0) { cout << "IMPOSSIBLE\n"; return 0; }
33     vector<int> route;
34     for (int v = n; v != 0; v = pre[v]) route.push_back(v);
35     reverse(route.begin(), route.end());
36     cout << route.size() << "\n";
37     for (int v : route) cout << v << " ";
38     cout << "\n";
39     return 0;
40 }

```

Complexity Analysis

Time Complexity. $O(n + m)$ – topological sort plus one relaxation sweep. **Space Complexity.** $O(n + m)$.

Key Takeaway

Longest path is intractable in general but linear on a DAG: relax edges in topological order and track predecessors. The acyclicity guarantee is the entire reason this DP is valid – the same guarantee Game Routes (next) exploits for path counting.

3.5 Game Routes

Problem Statement. Given a *directed acyclic* graph of n levels and m edges, count the number of distinct routes from level 1 to level n . Output the count modulo $10^9 + 7$.

Pattern Recognition

Counting paths in a **DAG** is a topological-order DP: the number of routes to a node is the sum of the routes to its predecessors. Acyclicity guarantees each predecessor's count is final before it is used.

Sum over predecessors

Let $ways[v]$ be the number of routes from 1 to v , with $ways[1] = 1$. In topological order, push each node's count forward: for every edge $u \rightarrow v$, add $ways[u]$ to $ways[v]$ (modulo $10^9 + 7$). Since u comes before v topologically, $ways[u]$ is complete when added. The answer is $ways[n]$.

Every Route Has a Unique Last Edge

Classifying routes to v by their final edge $u \rightarrow v$ partitions them without overlap, so summing $ways[u]$ over all predecessors u counts each route exactly once – the DAG analogue of the coin-counting recurrence.

Algorithm

1. Topologically sort. $ways[1] = 1$.
2. In topological order, for each edge $u \rightarrow v$ add $ways[u]$ to $ways[v]$ modulo $10^9 + 7$.
3. Output $ways[n]$.

Dry run

Dry Run. $1 \rightarrow 2, 1 \rightarrow 3, 2 \rightarrow 4, 3 \rightarrow 4; n = 4$.

- $ways[1] = 1, [2] = 1, [3] = 1, [4] = ways[2] + ways[3] = 2$.
- Two routes: $1 \rightarrow 2 \rightarrow 4$ and $1 \rightarrow 3 \rightarrow 4$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const long long MOD = 1e9 + 7;
4
5 int main() {
6     int n, m;
7     cin >> n >> m;

```

```

8     vector<vector<int>> adj(n + 1);
9     vector<int> indeg(n + 1, 0);
10    for (int i = 0; i < m; i++) {
11        int a, b; cin >> a >> b;
12        adj[a].push_back(b);
13        indeg[b]++;
14    }
15
16    vector<int> order, deg = indeg;
17    queue<int> q;
18    for (int i = 1; i <= n; i++) if (deg[i] == 0) q.push(i);
19    while (!q.empty()) {
20        int u = q.front(); q.pop(); order.push_back(u);
21        for (int v : adj[u]) if (--deg[v] == 0) q.push(v);
22    }
23
24    vector<long long> ways(n + 1, 0);
25    ways[1] = 1;
26    for (int u : order)
27        for (int v : adj[u])
28            ways[v] = (ways[v] + ways[u]) % MOD;
29
30    cout << ways[n] << "\n";
31    return 0;
32 }

```

Complexity Analysis

Time Complexity. $O(n + m)$. **Space Complexity.** $O(n + m)$.

Key Takeaway

DAG path counting sums predecessor counts in topological order. Longest Flight Route and Game Routes are the same sweep with $\max + 1$ versus $\text{sum} -$ the recurring pairing of optimisation and counting on the same DP skeleton.

3.6 Investigation

Problem Statement. Given a directed graph of n cities and m flights with non-negative prices, from city 1 to city n report four things: the minimum price; the number of minimum-price routes (mod $10^9 + 7$); and the minimum and maximum number of flights among minimum-price routes.

Pattern Recognition

All four answers come from a single **Dijkstra** augmented with three extra arrays alongside the distance: a route count, a min-edges count, and a max-edges count. Each is updated in lockstep with the distance relaxation.

Carry four quantities through relaxation

Maintain, per node: *dist* (min price), *cnt* (number of cheapest routes), *mn* (fewest flights on a cheapest route), *mx* (most flights on a cheapest route). When relaxing edge $u \rightarrow v$ with price w :

- If $\text{dist}[u] + w < \text{dist}[v]$: a strictly cheaper route – copy over $\text{cnt}[v] = \text{cnt}[u]$, $\text{mn}[v] = \text{mn}[u] + 1$, $\text{mx}[v] = \text{mx}[u] + 1$.

- If equal ($dist[u] + w = dist[v]$): another cheapest route – $cnt[v] += cnt[u]$, $mn[v] = \min(mn[v], mn[u] + 1)$, $mx[v] = \max(mx[v], mx[u] + 1)$.

One Pass, Four Aggregates

Because Dijkstra finalises nodes in nondecreasing distance, when a node is popped its *dist* is final and its three companion aggregates are complete. Updating them exactly on strict-improvement vs tie relaxations computes all four answers together.

Algorithm

1. Initialise arrays; $dist[1] = 0$, $cnt[1] = 1$, $mn[1] = mx[1] = 0$.
2. Run Dijkstra; on each edge apply the strict/tie update rules to all four arrays.
3. Output $dist[n]$, $cnt[n]$, $mn[n]$, $mx[n]$.

Dry run

Dry Run. $1 \rightarrow 2(1)$, $1 \rightarrow 3(1)$, $2 \rightarrow 4(1)$, $3 \rightarrow 4(1)$; $n = 4$.

- Cheapest price 2; two cheapest routes; both use 2 flights.
- Output 2 2 2 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const long long INF = 1e18, MOD = 1e9 + 7;
4
5 int main() {
6     int n, m;
7     cin >> n >> m;
8     vector<vector<pair<int, long long>>> adj(n + 1);
9     for (int i = 0; i < m; i++) {
10         int a, b; long long w; cin >> a >> b >> w;
11         adj[a].push_back({b, w});
12     }
13
14     vector<long long> dist(n + 1, INF), cnt(n + 1, 0);
15     vector<int> mn(n + 1, 1e9), mx(n + 1, 0);
16     dist[1] = 0; cnt[1] = 1; mn[1] = 0; mx[1] = 0;
17     priority_queue<pair<long long, int>, vector<pair<long long, int>>,
18         greater<>> pq;
19     pq.push({0, 1});
20     while (!pq.empty()) {
21         auto [d, u] = pq.top(); pq.pop();
22         if (d > dist[u]) continue;
23         for (auto& [v, w] : adj[u]) {
24             long long nd = d + w;
25             if (nd < dist[v]) {
26                 dist[v] = nd;
27                 cnt[v] = cnt[u];
28                 mn[v] = mn[u] + 1;
29                 mx[v] = mx[u] + 1;

```

```

30         pq.push({nd, v});
31     } else if (nd == dist[v]) {
32         cnt[v] = (cnt[v] + cnt[u]) % MOD;
33         mn[v] = min(mn[v], mn[u] + 1);
34         mx[v] = max(mx[v], mx[u] + 1);
35     }
36 }
37 }
38 cout << dist[n] << " " << cnt[n] << " " << mn[n] << " " << mx[n] <<
    "\n";
39 return 0;
40 }

```

Complexity Analysis

Time Complexity. $O((n + m) \log n)$. **Space Complexity.** $O(n + m)$.

Key Takeaway

Several statistics about shortest paths – count, min-edges, max-edges – piggyback on one Dijkstra by carrying extra aggregates updated on strict improvements versus ties. This “augment the relaxation” technique answers rich shortest-path questions in a single run.

3.7 Planets and Kingdoms

Problem Statement. Given a directed graph of n planets and m teleporters, partition the planets into kingdoms so that two planets are in the same kingdom iff each can reach the other. Output the number of kingdoms and each planet’s kingdom id.

Pattern Recognition

“Mutually reachable groups” are the **strongly connected components** (SCCs) of a directed graph. **Tarjan’s algorithm** finds all SCCs in a single DFS using discovery times and a low-link value that detects the root of each component.

Tarjan’s low-link

DFS assigns each node an increasing discovery index. A node’s *low* is the smallest index reachable from it using tree edges plus at most one back edge to a node still on the DFS stack. When a node’s *low* equals its own index, it is the root of an SCC: pop the DFS stack down to it, and those nodes form one component. Nodes not yet assigned sit on an explicit stack so cross-component edges are excluded.

low == index Marks a Component Root

The low-link captures “the earliest ancestor I can loop back to.” If a node cannot reach anything older than itself, it closes a strongly connected region, so everything above it on the stack is exactly that SCC.

Algorithm

1. DFS every node, maintaining discovery indices, low-links, and an on-stack marker.
2. When $low[u] = index[u]$, pop the stack to u , labelling those nodes with a new component id.

3. Output the component count and each node's id.

Dry run

Dry Run. $1 \rightarrow 2$, $2 \rightarrow 1$, $2 \rightarrow 3$.

- $\{1, 2\}$ are mutually reachable – one kingdom; $\{3\}$ alone.
- Two kingdoms.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, m, timer_ = 0, sccCount = 0;
5 vector<vector<int>> adj;
6 vector<int> disc, low, comp;
7 vector<bool> onStack;
8 stack<int> stk;
9
10 void tarjan(int u) {
11     disc[u] = low[u] = ++timer_;
12     stk.push(u); onStack[u] = true;
13     for (int v : adj[u]) {
14         if (!disc[v]) { tarjan(v); low[u] = min(low[u], low[v]); }
15         else if (onStack[v]) low[u] = min(low[u], disc[v]);
16     }
17     if (low[u] == disc[u]) { // u is an SCC root
18         sccCount++;
19         while (true) {
20             int x = stk.top(); stk.pop(); onStack[x] = false;
21             comp[x] = sccCount;
22             if (x == u) break;
23         }
24     }
25 }
26
27 int main() {
28     cin >> n >> m;
29     adj.assign(n + 1, {});
30     for (int i = 0; i < m; i++) { int a, b; cin >> a >> b; adj[a].
        push_back(b); }
31     disc.assign(n + 1, 0); low.assign(n + 1, 0);
32     comp.assign(n + 1, 0); onStack.assign(n + 1, false);
33
34     for (int i = 1; i <= n; i++) if (!disc[i]) tarjan(i);
35
36     cout << sccCount << "\n";
37     for (int i = 1; i <= n; i++) cout << comp[i] << " ";
38     cout << "\n";
39     return 0;
40 }

```

Complexity Analysis

Time Complexity. $O(n + m)$ – a single DFS. **Space Complexity.** $O(n + m)$; recursion depth $O(n)$ (use an iterative DFS for very large n).

Key Takeaway

Mutually-reachable partitions are SCCs, and Tarjan finds them in one DFS via the low-link “earliest reachable ancestor” test. SCCs underlie strong- connectivity checks, 2-SAT, and condensation DP – the next three problems.

3.8 Flight Routes Check

Problem Statement. Given a directed graph of n cities and m flights, determine whether you can travel between *every* pair of cities. If so output YES; otherwise output two cities a, b such that there is no route from a to b .

Pattern Recognition

Strong connectivity can be checked without full SCC decomposition: a graph is strongly connected iff, from any single node s , *everyone is reachable* in the original graph *and everyone can reach* s (reachable in the reversed graph). Two DFS runs suffice.

Forward and reverse reachability from one node

Pick $s = 1$. DFS in the original graph: if some node u is unreachable, then there is no route $1 \rightarrow u$, so print “1 u ”. Otherwise DFS in the graph with all edges reversed: if some node u is unreachable there, it cannot reach 1 in the original, so print “ u 1”. If both DFS runs cover every node, then 1 can reach all and all can reach 1, which makes every pair connected via 1 – output YES.

Strong Connectivity = Reach All and Be Reached from One Node

If a single node both reaches everyone and is reached by everyone, any two nodes a, b connect through it ($a \rightarrow s \rightarrow b$). So two reachability searches – one forward, one reversed – decide strong connectivity and produce a counterexample pair when it fails.

Algorithm

1. DFS/BFS from 1 in the original graph; if node u unreached, print “1 u ”.
2. DFS/BFS from 1 in the reversed graph; if node u unreached, print “ u 1”.
3. Else print YES.

Dry run

Dry Run. $1 \rightarrow 2, 2 \rightarrow 3$ (no way back); $n = 3$.

- Reversed DFS from 1 cannot reach 2 (edges reversed: $2 \rightarrow 1, 3 \rightarrow 2$), so 2 cannot reach 1.
- Output “2 1”.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void bfs(int s, vector<vector<int>>& g, vector<bool>& vis) {
5     queue<int> q; q.push(s); vis[s] = true;
6     while (!q.empty()) {
7         int u = q.front(); q.pop();
8         for (int v : g[u]) if (!vis[v]) { vis[v] = true; q.push(v); }
9     }
10 }
11
12 int main() {
13     int n, m;
14     cin >> n >> m;
15     vector<vector<int>> adj(n + 1), radj(n + 1);
16     for (int i = 0; i < m; i++) {
17         int a, b; cin >> a >> b;
18         adj[a].push_back(b);
19         radj[b].push_back(a);
20     }
21
22     vector<bool> vis(n + 1, false);
23     bfs(1, adj, vis);
24     for (int u = 1; u <= n; u++)
25         if (!vis[u]) { cout << 1 << " " << u << "\n"; return 0; }
26
27     fill(vis.begin(), vis.end(), false);
28     bfs(1, radj, vis);
29     for (int u = 1; u <= n; u++)
30         if (!vis[u]) { cout << u << " " << 1 << "\n"; return 0; }
31
32     cout << "YES\n";
33     return 0;
34 }

```

Complexity Analysis

Time Complexity. $O(n + m)$ – two traversals. **Space Complexity.** $O(n + m)$ for both adjacency lists.

Key Takeaway

Strong connectivity needs only two searches from one node – forward and on the reversed graph – not a full SCC pass. The unreachable node in either search is a ready-made counterexample pair, which is exactly what the problem asks for.

3.9 Giant Pizza

Problem Statement. There are n toppings and m friends. Each friend has two wishes, each either “topping t present (+)” or “topping t absent (−)”; a friend is satisfied if at least one wish holds. Decide the toppings so every friend is satisfied, or output IMPOSSIBLE.

Pattern Recognition

Each friend is a clause $(a \vee b)$ over boolean topping variables – this is **2-SAT**. Build the *implication graph* ($\neg a \Rightarrow b$ and $\neg b \Rightarrow a$), find its SCCs, and a variable is unsatisfiable iff it shares an SCC with its own negation.

Implications, SCCs, and the assignment

Represent each topping i by two literal-nodes: “present” and “absent,” with negation flipping between them. A clause $(a \vee b)$ adds $\neg a \rightarrow b$ and $\neg b \rightarrow a$ (if one wish fails the other must hold). Compute SCCs. If some literal and its negation land in the same SCC, they are forced equal – a contradiction – so the answer is **IMPOSSIBLE**. Otherwise assign each variable the truth value of whichever of its two literals appears *later* in topological order; with Tarjan’s reverse-topological numbering, that is the literal with the *smaller* component id.

A Literal and Its Negation Must Differ

If x and $\neg x$ are mutually reachable (same SCC), the constraints force $x = \neg x$, impossible. Otherwise picking the “downstream” (sink-ward) literal of each pair as true satisfies every implication, since truth flows along edges toward sinks.

Algorithm

1. Build the implication graph on $2n$ literal-nodes.
2. Find SCCs (Tarjan). If any variable has $comp[x] = comp[\neg x]$, print **IMPOSSIBLE**.
3. Else set topping i present iff $comp[present_i] < comp[absent_i]$; print $+/-$.

Dry run

Dry Run. One friend: $(+1 -2)$.

- Clause $(x_1 \vee \neg x_2)$: implications $\neg x_1 \rightarrow \neg x_2$ and $x_2 \rightarrow x_1$.
- An assignment like $x_1 = \text{present}$, $x_2 = \text{absent}$ satisfies it.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int N; // number of literal-
   nodes = 2n
5 vector<vector<int>> adj;
6 vector<int> disc, low, comp;
7 vector<bool> onStack;
8 stack<int> stk;
9 int timer_ = 0, sccCount = 0;
10
11 void tarjan(int u) {
12     disc[u] = low[u] = ++timer_;
13     stk.push(u); onStack[u] = true;
14     for (int v : adj[u]) {
15         if (!disc[v]) { tarjan(v); low[u] = min(low[u], low[v]); }

```

```

16         else if (onStack[v]) low[u] = min(low[u], disc[v]);
17     }
18     if (low[u] == disc[u]) {
19         sccCount++;
20         while (true) {
21             int x = stk.top(); stk.pop(); onStack[x] = false;
22             comp[x] = sccCount;
23             if (x == u) break;
24         }
25     }
26 }
27
28 int main() {
29     int n, m;
30     cin >> m >> n; // m friends, n
31     // toppings
32     N = 2 * n;
33     adj.assign(N, {});
34     auto lit = [&](char s, int t) { // literal node index
35         return 2 * (t - 1) + (s == '+' ? 0 : 1);
36     };
37     auto neg = [&](int l) { return l ^ 1; };
38     for (int i = 0; i < m; i++) {
39         char s1, s2; int t1, t2;
40         cin >> s1 >> t1 >> s2 >> t2;
41         int a = lit(s1, t1), b = lit(s2, t2);
42         adj[neg(a)].push_back(b); // clause (a OR b)
43         adj[neg(b)].push_back(a);
44     }
45     disc.assign(N, 0); low.assign(N, 0);
46     comp.assign(N, 0); onStack.assign(N, false);
47     for (int i = 0; i < N; i++) if (!disc[i]) tarjan(i);
48
49     string ans;
50     for (int i = 0; i < n; i++) {
51         int pres = 2 * i, abse = 2 * i + 1;
52         if (comp[pres] == comp[abse]) { cout << "IMPOSSIBLE\n"; return 0; }
53         ans += (comp[pres] < comp[abse]) ? '+' : '-'; // present iff
54         // smaller comp id
55     }
56     for (char c : ans) cout << c << " ";
57     cout << "\n";
58     return 0;
59 }

```

Complexity Analysis

Time Complexity. $O(n + m)$ – linear in the implication graph. **Space Complexity.** $O(n + m)$.

Key Takeaway

Any set of two-literal constraints is 2-SAT: build the implication graph, find SCCs, and check no variable shares an SCC with its negation. The sink-ward literal of each pair gives a satisfying assignment – one of the most elegant applications of strong connectivity.

3.10 Coin Collector

Problem Statement. Given a directed graph of n rooms (each holding some coins) and m corridors, you may start anywhere and walk along corridors (reusing rooms and corridors freely), collecting each room's coins once. Find the maximum coins collectible on a single walk.

Pattern Recognition

Inside a strongly connected component you can visit every room (cycle freely), so collect all its coins. **Condense each SCC into one super-node** carrying the component's total coins; the result is a DAG, and the answer is the maximum-weight path in it.

Condense, then longest path on the DAG

Find SCCs (Tarjan). Give each component a weight equal to the sum of its rooms' coins. Build the condensation: an edge $comp[u] \rightarrow comp[v]$ for every original edge crossing components. This graph is acyclic, so a longest-path DP applies: $dp[c] = weight[c] + \max(0, \max_{c \rightarrow c'} dp[c'])$. Because Tarjan numbers components in reverse topological order, condensation edges go from higher to lower ids, so processing components in increasing id order finalises successors first. The answer is the maximum dp over all components.

An SCC Is Free to Fully Explore

Once inside a cycle you can tour the whole SCC and grab every coin, so an SCC behaves like a single node worth its total coins. Condensation turns the messy cyclic graph into a DAG where longest path is easy.

Algorithm

1. Tarjan SCC; sum coins per component.
2. Build condensation edges (deduplication optional).
3. DP in increasing component id: $dp[c] = weight[c] + \max(0, \max dp[successor])$; answer is the maximum.

Dry run

Dry Run. $1 \rightarrow 2$, $2 \rightarrow 1$ (coins 2, 3), $2 \rightarrow 3$ (coins 5).

- SCC $\{1, 2\}$ worth 5; SCC $\{3\}$ worth 5; edge between them.
- Best walk collects $5 + 5 = 10$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int n, m, timer_ = 0, sccCount = 0;
5 vector<vector<int>> adj;
6 vector<long long> coin;
7 vector<int> disc, low, comp;
8 vector<bool> onStack;
9 stack<int> stk;

```

```

10
11 void tarjan(int u) {
12     disc[u] = low[u] = ++timer_;
13     stk.push(u); onStack[u] = true;
14     for (int v : adj[u]) {
15         if (!disc[v]) { tarjan(v); low[u] = min(low[u], low[v]); }
16         else if (onStack[v]) low[u] = min(low[u], disc[v]);
17     }
18     if (low[u] == disc[u]) {
19         sccCount++;
20         while (true) {
21             int x = stk.top(); stk.pop(); onStack[x] = false;
22             comp[x] = sccCount;
23             if (x == u) break;
24         }
25     }
26 }
27
28 int main() {
29     cin >> n >> m;
30     adj.assign(n + 1, {});
31     coin.assign(n + 1, 0);
32     for (int i = 1; i <= n; i++) cin >> coin[i];
33     vector<pair<int, int>> edges(m);
34     for (auto& [a, b] : edges) { cin >> a >> b; adj[a].push_back(b); }
35
36     disc.assign(n + 1, 0); low.assign(n + 1, 0);
37     comp.assign(n + 1, 0); onStack.assign(n + 1, false);
38     for (int i = 1; i <= n; i++) if (!disc[i]) tarjan(i);
39
40     vector<long long> weight(sccCount + 1, 0), dp(sccCount + 1, 0);
41     for (int i = 1; i <= n; i++) weight[comp[i]] += coin[i];
42     vector<vector<int>> cadj(sccCount + 1);
43     for (auto& [a, b] : edges)
44         if (comp[a] != comp[b]) cadj[comp[a]].push_back(comp[b]);
45
46     long long ans = 0;
47     for (int c = 1; c <= sccCount; c++) { // successors have
48         // smaller id
49         long long best = 0;
50         for (int nc : cadj[c]) best = max(best, dp[nc]);
51         dp[c] = weight[c] + best;
52         ans = max(ans, dp[c]);
53     }
54     cout << ans << "\n";
55     return 0;
56 }

```

Complexity Analysis

Time Complexity. $O(n + m)$ – SCC plus a DAG DP. **Space Complexity.** $O(n + m)$.

Key Takeaway

When cycles let you gather everything in a region, condense SCCs into weighted super-nodes and solve the resulting DAG. The “find SCCs → condense → DP on the DAG” pipeline is the standard way cyclic optimisation problems are made tractable.

Chapter 4

Successor and Functional Graphs

When every node has exactly one outgoing edge, the graph is a functional graph: following the arrows always leads into a cycle, with trees of nodes feeding into it. This rigid structure – “rho shaped” tails leading to loops – makes far-future queries answerable by binary lifting and cycle lengths computable in one pass.

4.1 One Outgoing Edge Changes Everything

In a successor graph each node x has a single successor $\text{succ}(x)$. Repeatedly applying succ from any start must eventually repeat a node and enter a cycle, since the state space is finite.

Functional-Graph Techniques

- **“Where am I after k steps?” for huge k – binary lifting:** precompute succ^{2^j} so any k is composed from its binary digits: Planets Queries I.
- **Distance between two nodes on the functional graph** – binary lifting plus cycle analysis (a directed analogue of LCA): Planets Queries II.
- **Length of the cycle each node eventually enters** – one pass marking tails and loops (or Floyd/Brent cycle finding): Planets Cycles.

Binary Lifting Answers k Steps in $O(\log k)$

Store, for each node, where you land after $2^0, 2^1, 2^2, \dots$ steps. Any k -step jump is the composition of the jumps for the set bits of k , so a query costs $O(\log k)$ after $O(n \log k)$ preprocessing – the same doubling idea as sparse tables and LCA.

4.2 Planets Queries I

Problem Statement. Each of n planets has exactly one teleporter to a successor planet. Answer q queries: starting from planet x and teleporting k times (where k can be up to 10^9), which planet do you reach?

Pattern Recognition

Following one edge per node many times is answered by **binary lifting**: precompute where you land after $2^0, 2^1, 2^2, \dots$ steps, then compose the jumps for the set bits of k to answer each query in $O(\log k)$.

Doubling the jump

Let $up[j][x]$ be the planet reached after 2^j teleports from x . The base is $up[0][x] = succ(x)$, and doubling gives $up[j][x] = up[j-1][up[j-1][x]]$ – jump 2^{j-1} , then 2^{j-1} more. For a query (x, k) , walk through the bits of k : for each set bit j , replace x with $up[j][x]$. After processing all bits, x is the destination.

Any k Is a Sum of Powers of Two

Because k decomposes into its binary digits, a k -step jump is the composition of the pre-computed 2^j -jumps for the bits set in k . Thus $O(n \log k)$ preprocessing yields $O(\log k)$ per query – the same doubling trick as sparse tables and LCA.

Algorithm

1. Read $succ$; build $up[0]$; fill $up[j]$ by composing the previous level.
2. For each (x, k) : for every set bit j of k , advance $x = up[j][x]$.
3. Output the final x .

Dry run

Dry Run. $succ = [2, 3, 1]$ (a 3-cycle); query $x = 1, k = 4$.

- $k = 4 = 100_2$: jump $2^2 = 4$ steps. $up[2][1]$ = four steps around the 3-cycle = $1 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 2$, landing on 2.
- Answer 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int LOG = 30; // 2^30 > 1e9
4
5 int main() {
6     int n, q;
7     cin >> n >> q;
8     vector<vector<int>> up(LOG, vector<int>(n + 1));
9     for (int i = 1; i <= n; i++) cin >> up[0][i]; // successor
10    for (int j = 1; j < LOG; j++)
11        for (int i = 1; i <= n; i++)
12            up[j][i] = up[j - 1][up[j - 1][i]];

```

```

13
14     while (q--) {
15         int x, k;
16         cin >> x >> k;
17         for (int j = 0; j < LOG; j++)
18             if (k & (1 << j)) x = up[j][x];           // apply 2^j-jump
19             for set bits
20         cout << x << "\n";
21     }
22     return 0;
23 }

```

Complexity Analysis

Time Complexity. $O(n \log k)$ preprocessing, $O(\log k)$ per query. **Space Complexity.** $O(n \log k)$ for the lifting table.

Key Takeaway

Binary lifting answers “ k steps ahead” in a functional graph in logarithmic time by composing power-of-two jumps. The same table underlies k -th ancestor queries and LCA, and powers the harder Planets Queries II next.

4.3 Planets Queries II

Problem Statement. Each of n planets has one teleporter. Answer q queries: the minimum number of teleports to go from planet a to planet b , or -1 if impossible.

Pattern Recognition

A functional graph is “rho-shaped”: trees of tail nodes feeding into cycles. Decompose it into cycles and the depth of each node to its cycle, then a query splits into a **tail case** (answered by binary lifting) and a **cycle case** (answered by modular arithmetic on the cycle).

Tails into cycles

Peel nodes of in-degree 0 repeatedly; whatever remains lies on cycles. Label each cycle, giving its nodes positions $0, 1, \dots$ and recording the cycle length. Each tail node gets a *depth* (steps to reach its cycle) and an *entry* (the cycle node it first lands on). For a query (a, b) :

- **Tail case:** if b lies on the path from a into the cycle, then $\text{depth}[a] \geq \text{depth}[b]$ and jumping $\text{depth}[a] - \text{depth}[b]$ steps (binary lifting) from a lands on b ; that jump count is the answer.
- **Cycle case:** if b is on the same cycle a reaches, walk $\text{depth}[a]$ steps to a ’s entry, then around the cycle: $\text{depth}[a] + ((\text{pos}[b] - \text{pos}[\text{entry}[a]]) \bmod L)$.

Take whichever applies; otherwise -1 .

Split the Query by Where b Lives

The rho structure means b is reachable from a only if it is on a ’s tail before the cycle, or on the cycle a enters. Binary lifting verifies the tail case exactly; modular position arithmetic handles the cycle case.

Algorithm

1. Build binary lifting for the successor function.
2. Peel to find cycle nodes; assign cycle ids, positions, lengths; compute *depth* and *entry* for tail nodes.
3. Per query, evaluate the tail and cycle cases; output the minimum or -1 .

Dry run

Dry Run. $\text{succ} = [2, 3, 1, 3]$: $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$ cycle, $4 \rightarrow 3$ tail.

- Query (4, 1): 4 enters the cycle at 3 in 1 step, then $3 \rightarrow 1$ is 2 more steps around the cycle. Answer 3.
- Query (1, 4): 4 is a tail node not reachable from the cycle -1 .

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const int LOG = 20;
4
5 int main() {
6     int n, q;
7     cin >> n >> q;
8     vector<int> succ(n + 1);
9     vector<vector<int>> up(LOG, vector<int>(n + 1));
10    vector<int> indeg(n + 1, 0);
11    for (int i = 1; i <= n; i++) { cin >> succ[i]; up[0][i] = succ[i];
12        indeg[succ[i]]++; }
13    for (int j = 1; j < LOG; j++)
14        for (int i = 1; i <= n; i++) up[j][i] = up[j - 1][up[j - 1][i]];
15
16    auto jump = [&](int x, int k) {
17        for (int j = 0; j < LOG; j++) if (k & (1 << j)) x = up[j][x];
18        return x;
19    };
20
21    // peel tree nodes; remaining are cycle nodes
22    vector<int> deg = indeg, order;
23    queue<int> pq;
24    for (int i = 1; i <= n; i++) if (deg[i] == 0) pq.push(i);
25    vector<bool> removed(n + 1, false);
26    while (!pq.empty()) {
27        int u = pq.front(); pq.pop();
28        removed[u] = true; order.push_back(u);
29        if (--deg[succ[u]] == 0) pq.push(succ[u]);
30    }
31
32    vector<int> depth(n + 1, 0), entry(n + 1, 0), lead(n + 1, 0), pos(n + 1, 0);
33    vector<bool> isCyc(n + 1, false);
34    vector<int> cycLen(n + 1, 0);
35    int cid = 0;
36    for (int i = 1; i <= n; i++) {

```

```

36     if (removed[i] || isCyc[i]) continue;
37     cid++; int c = i, len = 0;
38     for (int x = i; !isCyc[x]; x = succ[x]) {
39         isCyc[x] = true; pos[x] = len++; lead[x] = cid;
40         entry[x] = x; depth[x] = 0;
41     }
42     cycLen[cid] = len;
43 }
44 for (int idx = (int)order.size() - 1; idx >= 0; idx--) {    //
45     reverse peel order
46     int u = order[idx];
47     depth[u] = depth[succ[u]] + 1;
48     entry[u] = entry[succ[u]];
49     lead[u] = lead[succ[u]];
50 }
51 while (q--) {
52     int a, b;
53     cin >> a >> b;
54     long long ans = -1;
55     if (a == b) { cout << 0 << "\n"; continue; }
56     // tail case: b on a's path into the cycle
57     if (depth[a] >= depth[b]) {
58         int steps = depth[a] - depth[b];
59         if (jump(a, steps) == b) ans = steps;
60     }
61     // cycle case: b on the same cycle a reaches
62     if (isCyc[b] && lead[b] == lead[a]) {
63         int L = cycLen[lead[a]];
64         long long steps = depth[a] + ((pos[b] - pos[entry[a]]) % L
65             + L) % L;
66         if (ans == -1 || steps < ans) ans = steps;
67     }
68     cout << ans << "\n";
69 }
70 return 0;
71 }

```

Complexity Analysis

Time Complexity. $O(n \log n)$ preprocessing, $O(\log n)$ per query. **Space Complexity.** $O(n \log n)$ for the lifting table.

Key Takeaway

Distances in a functional graph split by structure: binary lifting settles the tree-tail case, modular cycle arithmetic settles the cycle case. Seeing the rho shape – tails feeding cycles – is the key to organising the query logic.

4.4 Planets Cycles

Problem Statement. Each of n planets has one teleporter. For each starting planet, output how many planets you visit before returning to a planet you have already seen (i.e. the number of teleports until the first repeat).

Pattern Recognition

In a functional graph, starting anywhere you walk a tail and then loop a cycle. The count until the first repeat is $depth + cycle\ length$: the tail steps to reach the cycle plus the cycle's size. So the same rho decomposition as Planets Queries II answers every node at once.

Tail length plus cycle length

Peel to identify cycle nodes; for each cycle record its length. A node on a cycle simply revisits after $cycleLen$ steps. A tail node walks $depth$ steps to reach the cycle, then $cycleLen$ more to return to its entry point – so its answer is $depth + cycleLen(entry)$. Computing $depth$ and the cycle each node leads to (in reverse peel order) gives all answers in one pass.

Every Walk Is Tail Then Loop

The first repeat is always the cycle entry, reached again after one full loop. Hence the visited count is exactly tail length plus cycle length, independent of where on the cycle you enter.

Algorithm

1. Peel tree nodes; label cycles with their lengths.
2. Cycle node answer = $cycleLen$; tail node answer = $depth + cycleLen(lead)$.
3. Output every node's answer.

Dry run

Dry Run. $succ = [2, 3, 1, 3]$: cycle $1 \rightarrow 2 \rightarrow 3$ (length 3), tail $4 \rightarrow 3$.

- Nodes 1, 2, 3: answer 3 each. Node 4: $depth\ 1 + cycle\ 3 = 4$.
- Output 3 3 3 4.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     vector<int> succ(n + 1), indeg(n + 1, 0);
8     for (int i = 1; i <= n; i++) { cin >> succ[i]; indeg[succ[i]]++; }
9
10    vector<int> deg = indeg, order;
11    queue<int> pq;
12    for (int i = 1; i <= n; i++) if (deg[i] == 0) pq.push(i);
13    vector<bool> removed(n + 1, false);
14    while (!pq.empty()) {
15        int u = pq.front(); pq.pop();
16        removed[u] = true; order.push_back(u);
17        if (--deg[succ[u]] == 0) pq.push(succ[u]);
18    }
19
20    vector<int> depth(n + 1, 0), lead(n + 1, 0), cycLen(n + 1, 0);

```

```

21     vector<bool> isCyc(n + 1, false);
22     int cid = 0;
23     for (int i = 1; i <= n; i++) {
24         if (removed[i] || isCyc[i]) continue;
25         cid++; int len = 0;
26         for (int x = i; !isCyc[x]; x = succ[x]) { isCyc[x] = true; lead
27             [x] = cid; len++; }
28         cycLen[cid] = len;
29     }
30     for (int idx = (int)order.size() - 1; idx >= 0; idx--) {
31         int u = order[idx];
32         depth[u] = depth[succ[u]] + 1;
33         lead[u] = lead[succ[u]];
34     }
35     for (int i = 1; i <= n; i++)
36         cout << depth[i] + cycLen[lead[i]] << " ";
37     cout << "\n";
38     return 0;
39 }

```

Complexity Analysis

Time Complexity. $O(n)$ – peeling plus one reverse pass. **Space Complexity.** $O(n)$.

Key Takeaway

Steps-until-repeat in a functional graph is tail depth plus cycle length, computed for every node from one rho decomposition. Recognising the tail- then-loop structure turns a per-node simulation into a single linear pass.

Chapter 5

Minimum Spanning Trees

To connect every node at least total cost, greedily add the cheapest edge that joins two so-far-separate parts. The Disjoint Set data structure makes “are these already connected?” almost free, and the same structure – run in reverse, or watched over time – answers dynamic connectivity questions too.

5.1 Greedy Connection with Disjoint Set

A minimum spanning tree (MST) connects all n nodes with $n-1$ edges of minimum total weight. Kruskal’s algorithm sorts edges and adds each that does not form a cycle, using Union-Find for the cycle test.

MST and Connectivity Problems

- **Cheapest way to connect everything – Kruskal’s MST** with Disjoint Set: Road Reparation (report impossible if the graph is disconnected).
- **Add edges over time, tracking components and the largest component – incremental Disjoint Set:** Road Construction.

Union-Find Makes the Cycle Test $O(1)$

Kruskal’s correctness rests on always taking the cheapest edge between two different components. Disjoint Set answers “same component?” in near constant time (with path compression and union by size), and its size counter directly gives the largest-component answer for Road Construction.

5.2 Road Reparation

Problem Statement. Given n cities and m possible roads, each with a repair cost, choose roads of minimum total cost so that every city is reachable from every other. Output the minimum cost, or IMPOSSIBLE if the cities cannot all be connected.

Pattern Recognition

“Connect all cities at minimum total cost” is a **minimum spanning tree**. **Kruskal’s algorithm** sorts roads by cost and adds each that joins two different components, using Disjoint Set for the cycle test; if fewer than $n - 1$ roads are added, the graph is disconnected.

Kruskal with Union-Find

Sort all roads ascending by cost. Process each: if its endpoints are in different components (checked via Disjoint Set), add it to the MST, union the components, and accumulate the cost. Skip roads whose endpoints are already connected (they would form a cycle). After processing, if exactly $n - 1$ roads were added, output the total; otherwise the graph could not be fully connected, so print IMPOSSIBLE.

Cheapest Safe Edge First

The MST is built greedily: the cheapest edge crossing between two components is always safe to include. Disjoint Set makes “same component?” near-constant, and reaching $n - 1$ edges certifies a spanning tree exists.

Algorithm

1. Sort roads by cost. Initialise Disjoint Set.
2. For each road, if it joins two components, union and add its cost; count added roads.
3. If $n - 1$ roads were added, output the total cost; else IMPOSSIBLE.

Dry run

Dry Run. $n = 3$; roads 1–2(3), 2–3(5), 1–3(4).

- Sorted: 3, 4, 5. Add 1–2(3); add 1–3(4); skip 2–3 (cycle).
- Total = 3 + 4 = 7, two roads = $n - 1$: connected.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct DSU {
5     vector<int> parent, sz;
6     DSU(int n) : parent(n + 1), sz(n + 1, 1) { iota(parent.begin(),
7         parent.end(), 0); }
8     int find(int x) { return parent[x] == x ? x : parent[x] = find(
9         parent[x]); }
10    bool unite(int a, int b) {
11        a = find(a); b = find(b);
12        if (a == b) return false;

```

```

11         if (sz[a] < sz[b]) swap(a, b);
12         parent[b] = a; sz[a] += sz[b];
13         return true;
14     }
15 };
16
17 int main() {
18     int n, m;
19     cin >> n >> m;
20     vector<tuple<long long, int, int>> edges(m);
21     for (auto& [w, a, b] : edges) cin >> a >> b >> w;
22     sort(edges.begin(), edges.end());
23
24     DSU dsu(n);
25     long long total = 0;
26     int used = 0;
27     for (auto& [w, a, b] : edges)
28         if (dsu.unite(a, b)) { total += w; used++; }
29
30     if (used == n - 1) cout << total << "\n";
31     else cout << "IMPOSSIBLE\n";
32     return 0;
33 }

```

Complexity Analysis

Time Complexity. $O(m \log m)$ – sorting dominates; unions are near-constant. **Space Complexity.** $O(n + m)$.

Key Takeaway

Minimum-cost full connection is an MST; Kruskal plus Disjoint Set builds it and, by counting added edges, detects disconnection. This is the canonical network-design primitive.

5.3 Road Construction

Problem Statement. There are n cities and initially no roads. Roads are built one at a time (m of them). After each new road, output the current number of connected components and the size of the largest component.

Pattern Recognition

Adding edges and tracking connectivity over time is **incremental Disjoint Set**. Maintain a running component count and the maximum component size, both updated in $O(\alpha)$ whenever a union actually merges two different components.

Update counts on each real union

Start with n components, each of size 1, and a maximum size of 1. For each new road, union its endpoints: if they were already in the same component, nothing changes. If they were separate, the component count drops by one, the merged component's size is the sum of the two, and the maximum size is updated against it. Print the count and maximum after each road.

Only Real Merges Change the Answer

Union-Find's size counter gives the merged component's size immediately, so the largest-component tracker updates in $O(1)$ per successful union. A union that finds both endpoints already together leaves both statistics untouched.

Algorithm

1. Disjoint Set of n singletons; $components = n$, $maxSize = 1$.
2. Per road: if the endpoints unite, decrement $components$ and update $maxSize$ with the new size.
3. Output $components$ and $maxSize$ after each road.

Dry run

Dry Run. $n = 4$; roads 1–2, 3–4, 2–3.

- After 1–2: components 3, max 2.
- After 3–4: components 2, max 2.
- After 2–3: merges the two pairs – components 1, max 4.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int n, m;
6      cin >> n >> m;
7      vector<int> parent(n + 1), sz(n + 1, 1);
8      iota(parent.begin(), parent.end(), 0);
9      function<int(int)> find = [&](int x) { return parent[x] == x ? x :
      parent[x] = find(parent[x]); };
10
11     int components = n, maxSize = 1;
12     while (m--) {
13         int a, b; cin >> a >> b;
14         a = find(a); b = find(b);
15         if (a != b) {
16             if (sz[a] < sz[b]) swap(a, b);
17             parent[b] = a;
18             sz[a] += sz[b];
19             components--;
20             maxSize = max(maxSize, sz[a]);
21         }
22         cout << components << " " << maxSize << "\n";
23     }
24     return 0;
25 }
```

Complexity Analysis

Time Complexity. $O((n + m) \alpha(n))$ – near-linear. **Space Complexity.** $O(n)$.

Key Takeaway

Online connectivity under edge additions is incremental Union-Find, with the component count and largest-size maintained on each real merge. Disjoint Set handles edge *additions* gladly; edge *deletions* are the hard case needing offline or advanced structures.

Chapter 6

Eulerian and Hamiltonian Paths

Two famous path problems look similar but could not be more different in difficulty. An Eulerian path uses every edge exactly once and exists under a simple degree condition, findable in linear time. A Hamiltonian path visits every node exactly once, is NP-hard in general, and here is tamed only by small n (bitmask DP) or a heuristic on a grid.

6.1 Every Edge Once vs Every Node Once

The distinction is the whole chapter: edges (Eulerian, easy) versus nodes (Hamiltonian, hard).

Which Path Are You Asked For?

- **Use every edge exactly once – Eulerian circuit/path** via Hierholzer’s algorithm, after checking the degree conditions: Mail Delivery (undirected circuit), Teleporters Path (directed path), De Bruijn Sequence (Eulerian path in a De Bruijn graph).
- **Visit every node exactly once (count or find) – Hamiltonian:** bitmask DP over visited sets when $n \leq 20$ (Hamiltonian Flights), or a Warnsdorff heuristic on a board (Knight’s Tour).

Eulerian Existence Is a Degree Check

An undirected connected graph has an Eulerian circuit iff every vertex has even degree; a directed one iff every vertex’s in-degree equals its out-degree (for a path, exactly one $+1/-1$ imbalance pair). Hierholzer’s algorithm then stitches edge-disjoint cycles into a single trail in linear time.

6.2 Mail Delivery

Problem Statement. Given an undirected graph of n nodes and m edges, find a route that starts and ends at node 1 and traverses *every edge exactly once* (an Eulerian circuit), or output IMPOSSIBLE.

Pattern Recognition

An **Eulerian circuit** exists iff the graph is connected (over edge-bearing nodes) and *every vertex has even degree*. When it exists, **Hierholzer's algorithm** stitches it together in linear time.

Degree check, then Hierholzer

First verify every vertex has even degree and all edges lie in one connected piece reachable from node 1; otherwise no circuit exists. Hierholzer's then walks edges greedily from node 1, marking each used, until it returns to a node with no unused edges – backtracking (popping to an output list) whenever stuck, which merges sub-cycles into one trail. Reversing the popped order yields the Eulerian circuit. A per-node pointer skips already-used edges so total work stays linear.

Even Degrees Guarantee You Can Always Leave

Entering a vertex uses one incident edge; an even degree guarantees an unused edge to leave by, so the walk only gets stuck back at the start. Hierholzer exploits this to splice every local cycle into a single circuit.

Algorithm

1. Check all degrees even and edges connected; else IMPOSSIBLE.
2. Hierholzer from node 1 using a stack and per-node edge pointer, marking edges used.
3. Reverse the popped nodes; if it uses all m edges, print it.

Dry run

Dry Run. Square 1–2, 2–3, 3–4, 4–1.

- All degrees 2 (even), connected – a circuit exists.
- Hierholzer yields $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;
7     vector<vector<pair<int,int>>> adj(n + 1);    // {neighbor, edgeId}
8     vector<int> deg(n + 1, 0);
9     for (int i = 0; i < m; i++) {
10         int a, b; cin >> a >> b;
11         adj[a].push_back({b, i});
12         adj[b].push_back({a, i});
    }

```

```

13     deg[a]++; deg[b]++;
14 }
15 for (int i = 1; i <= n; i++)
16     if (deg[i] % 2 != 0) { cout << "IMPOSSIBLE\n"; return 0; }
17
18 vector<int> ptr(n + 1, 0);
19 vector<bool> used(m, false);
20 vector<int> circuit;
21 stack<int> st;
22 st.push(1);
23 while (!st.empty()) {
24     int u = st.top();
25     while (ptr[u] < (int)adj[u].size() && used[adj[u][ptr[u]].
26         second]) ptr[u]++;
27     if (ptr[u] == (int)adj[u].size()) { circuit.push_back(u); st.
28         pop(); }
29     else {
30         auto [v, id] = adj[u][ptr[u]];
31         used[id] = true;
32         st.push(v);
33     }
34 }
35 reverse(circuit.begin(), circuit.end());
36 if ((int)circuit.size() != m + 1) { cout << "IMPOSSIBLE\n"; return
37     0; }
38 for (int v : circuit) cout << v << " ";
39 cout << "\n";
40 return 0;
41 }

```

Complexity Analysis

Time Complexity. $O(n + m)$ – each edge is used once; pointers avoid rescanning. **Space Complexity.** $O(n + m)$.

Key Takeaway

Eulerian circuits are a degree check plus Hierholzer's stitching of cycles. The per-node pointer and edge-used markers keep it linear even with multi-edges. The circuit-size check at the end catches disconnection.

6.3 De Bruijn Sequence

Problem Statement. For a given n , output a binary string that contains *every* binary string of length n as a substring, and is as short as possible (length $2^n + n - 1$, treating the string as almost cyclic).

Pattern Recognition

A De Bruijn sequence is an **Eulerian path in the De Bruijn graph**: nodes are the 2^{n-1} binary strings of length $n - 1$, and each length- n string is an edge from its $(n - 1)$ -prefix to its $(n - 1)$ -suffix. Every node has equal in- and out-degree, so an Eulerian circuit exists.

Edges are the length- n strings

Build a graph whose nodes are all $(n - 1)$ -bit strings. For each n -bit string, add an edge from its first $n - 1$ bits to its last $n - 1$ bits, labelled by the last bit. Because appending 0 or 1 to any node gives a valid edge, in-degree equals out-degree everywhere – the Eulerian condition. Run Hierholzer to traverse every edge once; concatenating the starting node with each edge's appended bit produces a string in which every length- n substring appears exactly once.

Traverse Every Length- n String as an Edge

Walking an Eulerian circuit uses each length- n string (edge) exactly once, and consecutive edges overlap in $n - 1$ characters, so the emitted bits form a compact sequence containing all 2^n patterns.

Algorithm

1. Nodes = integers $0 \dots 2^{n-1} - 1$; for each node v and bit b , edge to $((v \ll 1) | b) \bmod 2^{n-1}$ labelled b .
2. Hierholzer from node 0, recording edge labels.
3. Output the start node's $n - 1$ bits followed by the recorded labels.

Dry run

Dry Run. $n = 2$: nodes $\{0, 1\}$ (1-bit), edges for 00, 01, 10, 11.

- Eulerian circuit visits all four length-2 strings.
- A valid output is 00110: substrings 00, 01, 11, 10 all appear.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n;
6     cin >> n;
7     if (n == 1) { cout << "01\n"; return 0; }
8
9     int nodes = 1 << (n - 1);
10    int mask = nodes - 1;
11    vector<int> ptr(nodes, 0);           // next bit to try
12                                       // per node (0 then 1)
13
14    // Hierholzer recording edge labels; stack holds {node, label used
15    // to enter}
16    vector<int> labels;
17    stack<pair<int, int>> st;
18    st.push({0, -1});                  // start at node 0,
19                                       // no entering label
20    while (!st.empty()) {
21        auto [v, lab] = st.top();
22        if (ptr[v] < 2) {               // an unused out-
23            edge (bit 0 then 1)
24            int b = ptr[v]++;

```

```

21         int to = ((v << 1) | b) & mask;
22         st.push({to, b});
23     } else {
24         st.pop();
25         if (lab != -1) labels.push_back(lab);
26     }
27 }
28 reverse(labels.begin(), labels.end());
29
30 string res(n - 1, '0'); // start node = (n
    -1) zero bits
31 for (int b : labels) res += char('0' + b);
32 cout << res << "\n";
33 return 0;
34 }

```

Complexity Analysis

Time Complexity. $O(2^n)$ – one traversal of the 2^n edges. **Space Complexity.** $O(2^n)$ for the recursion/circuit.

Key Takeaway

A De Bruijn sequence is an Eulerian circuit in the De Bruijn graph, where length- n strings are edges over $(n - 1)$ -length nodes. The construction is the flagship application of Eulerian paths beyond literal edge-traversal puzzles.

6.4 Teleporters Path

Problem Statement. Given a *directed* graph of n nodes and m edges, find a path from node 1 to node n that uses *every edge exactly once* (a directed Eulerian path), or output IMPOSSIBLE.

Pattern Recognition

A directed **Eulerian path** from 1 to n exists iff node 1 has one more outgoing than incoming edge, node n one more incoming than outgoing, every other node has equal in- and out-degree, and all edges are connected. **Hierholzer's algorithm** then constructs it.

Degree imbalance pins the endpoints

Check the directed degree conditions: $out[1] - in[1] = 1$, $in[n] - out[n] = 1$, and $in = out$ for all other nodes; also confirm every edge is reachable so the trail is a single path. Then run Hierholzer from node 1: follow unused out-edges via a per-node pointer, pushing onto a stack; when a node has no unused out-edges, pop it to the path. Reversing the popped nodes gives the Eulerian path, which necessarily ends at n .

The +1/-1 Imbalance Fixes Start and End

Equal in/out degree everywhere would force a circuit; a single node with an extra out-edge must be the start and a single node with an extra in-edge the end. Hierholzer respects this automatically, terminating the trail at n .

Algorithm

1. Compute in/out degrees; verify the $1 \rightarrow n$ Eulerian-path conditions, else IMPOSSIBLE.
2. Hierholzer from node 1 over directed out-edges with a per-node pointer.
3. Reverse the popped nodes; if all m edges are used and it ends at n , print it.

Dry run

Dry Run. $1 \rightarrow 2$, $2 \rightarrow 3$, $1 \rightarrow 3$? (degrees must fit).

- With $1 \rightarrow 2$, $2 \rightarrow 3$: $out[1] - in[1] = 1$, $in[3] - out[3] = 1$, node 2 balanced.
- Path $1 \rightarrow 2 \rightarrow 3$ uses both edges.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n, m;
6     cin >> n >> m;
7     vector<vector<int>> adj(n + 1);
8     vector<int> indeg(n + 1, 0), outdeg(n + 1, 0);
9     for (int i = 0; i < m; i++) {
10         int a, b; cin >> a >> b;
11         adj[a].push_back(b);
12         outdeg[a]++; indeg[b]++;
13     }
14
15     // Eulerian path 1 -> n conditions
16     bool ok = (outdeg[1] - indeg[1] == 1) && (indeg[n] - outdeg[n] ==
17         1);
18     for (int i = 1; i <= n && ok; i++)
19         if (i != 1 && i != n && indeg[i] != outdeg[i]) ok = false;
20     if (!ok) { cout << "IMPOSSIBLE\n"; return 0; }
21
22     vector<int> ptr(n + 1, 0), path;
23     stack<int> st;
24     st.push(1);
25     while (!st.empty()) {
26         int u = st.top();
27         if (ptr[u] < (int)adj[u].size()) {
28             int v = adj[u][ptr[u]++];
29             st.push(v);
30         } else { path.push_back(u); st.pop(); }
31     }
32     reverse(path.begin(), path.end());
33
34     if ((int)path.size() != m + 1 || path.front() != 1 || path.back()
35         != n) {
36         cout << "IMPOSSIBLE\n"; return 0;
37     }
38     for (int v : path) cout << v << " ";
39     cout << "\n";
40     return 0;

```

39 }

Complexity Analysis**Time Complexity.** $O(n + m)$. **Space Complexity.** $O(n + m)$.**Key Takeaway**

Directed Eulerian paths need the in/out degree imbalance to name the endpoints, then Hierholzer. The final check – all edges used and the trail runs $1 \rightarrow n$ – guards against disconnected edge sets that pass the local degree test.

6.5 Hamiltonian Flights

Problem Statement. Given a directed graph of n cities ($n \leq 20$) and m flights, count the number of routes from city 1 to city n that visit *every* city exactly once (Hamiltonian paths). Output the count modulo $10^9 + 7$.

Pattern Recognition

Hamiltonian path counting is NP-hard in general, but $n \leq 20$ invites **bitmask DP**: the state is (set of visited cities, current city), and $2^n \cdot n$ states are affordable.

DP over visited subsets

Let $dp[mask][v]$ be the number of routes that start at city 1, have visited exactly the set $mask$ (which must include 1 and v), and currently sit at v . Base case: $dp[\{1\}][1] = 1$. Transition: from state $(mask, u)$, for each flight $u \rightarrow v$ with $v \notin mask$, add $dp[mask][u]$ to $dp[mask \cup \{v\}][v]$. The answer is $dp[full][n]$ – all cities visited, ending at n .

The Visited Set Is the State

A Hamiltonian path cannot revisit a city, so the exact set already visited (plus where you are) fully determines valid continuations. Encoding that set as a bitmask makes the DP finite: 2^n subsets times n endpoints.

Algorithm

1. $dp[1 \ll 0][1] = 1$ (city 1 is bit 0).
2. For each mask (increasing) and each city u in it with $dp > 0$: extend along flights $u \rightarrow v$ with v unvisited.
3. Output $dp[(1 \ll n) - 1][n - 1]$.

Dry run

Dry Run. $n = 3$; flights $1 \rightarrow 2$, $2 \rightarrow 3$, $1 \rightarrow 3$.

- Only $1 \rightarrow 2 \rightarrow 3$ visits all three ending at 3; $1 \rightarrow 3$ skips city 2.
- Count = 1.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 const long long MOD = 1e9 + 7;
4
5 int main() {
6     int n, m;
7     cin >> n >> m;
8     vector<vector<int>> adj(n); // 0-indexed cities
9     for (int i = 0; i < m; i++) {
10         int a, b; cin >> a >> b;
11         adj[a - 1].push_back(b - 1);
12     }
13
14     int full = 1 << n;
15     vector<vector<long long>> dp(full, vector<long long>(n, 0));
16     dp[1][0] = 1; // visited {city 0},
17                  // at city 0
18     for (int mask = 1; mask < full; mask++)
19         for (int u = 0; u < n; u++) {
20             if (!(mask & (1 << u)) || dp[mask][u] == 0) continue;
21             for (int v : adj[u])
22                 if (!(mask & (1 << v)))
23                     dp[mask | (1 << v)][v] =
24                         (dp[mask | (1 << v)][v] + dp[mask][u]) % MOD;
25         }
26     cout << dp[full - 1][n - 1] << "\n";
27     return 0;
28 }

```

Complexity Analysis

Time Complexity. $O(2^n \cdot n^2)$ in the worst case (n endpoints, n neighbours). **Space Complexity.** $O(2^n \cdot n)$ for the DP table.

Key Takeaway

Hamiltonian counting is bitmask DP with state (visited set, current node); small n is what makes the exponential state space tractable. This subset-DP template also solves the travelling salesman problem and assignment-style problems.

6.6 Knight's Tour

Problem Statement. On an 8×8 chessboard, starting from a given square, find a knight's tour: a sequence of knight moves visiting every square exactly once. Output the board with each square labelled by the step at which the knight visits it.

Pattern Recognition

A knight's tour is a Hamiltonian path on the knight-move graph. Rather than exponential search, **Warnsdorff's heuristic** makes it near-instant: always move to the square with the *fewest* onward unvisited moves.

Warnsdorff's rule

From the current square, consider all legal knight moves to unvisited squares. Move to the one whose *onward degree* – the number of unvisited squares *it* can reach – is smallest. The intuition: squares with few exits are the easiest to get stranded in, so visit them early while they are still reachable. Ties can be broken arbitrarily (or by a fixed order). On a standard board this greedy choice completes a full tour without backtracking in practice.

Escape the Tightest Corners First

Choosing the least-accessible next square keeps the most flexible squares for later, avoiding dead ends. This “most-constrained first” heuristic turns an exponential Hamiltonian search into an $O(64)$ greedy walk.

Algorithm

1. Mark the start square as step 1.
2. Repeat 63 times: among unvisited knight-move targets, pick the one with the fewest unvisited onward moves; go there, label the next step.
3. Output the 8×8 grid of step labels.

Dry run

Dry Run. Start at a corner, e.g. $(0, 0)$.

- The corner has only two moves; Warnsdorff steers toward low-degree squares first, threading all 64 cells.
- The result is a valid tour numbered $1 \dots 64$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int dr[] = {-2,-2,-1,-1,1,1,2,2};
5  int dc[] = {-1,1,-2,2,-2,2,-1,1};
6
7  int main() {
8      int sr, sc;
9      cin >> sc >> sr;                // column, row (1-
10         indexed)
11     sr--; sc--;
12
13     vector<vector<int>> board(8, vector<int>(8, 0));
14     auto inb = [](int r, int c) { return r >= 0 && r < 8 && c >= 0 && c
15         < 8; };
16     auto onward = [&](int r, int c) {                // unvisited onward
17         moves
18         int cnt = 0;
19         for (int k = 0; k < 8; k++) {
20             int nr = r + dr[k], nc = c + dc[k];
21             if (inb(nr, nc) && board[nr][nc] == 0) cnt++;
22         }
23         return cnt;
24     };

```

```

21     };
22
23     int r = sr, c = sc;
24     board[r][c] = 1;
25     for (int step = 2; step <= 64; step++) {
26         int best = -1, bestR = -1, bestC = -1;
27         for (int k = 0; k < 8; k++) {
28             int nr = r + dr[k], nc = c + dc[k];
29             if (!inb(nr, nc) || board[nr][nc]) continue;
30             int deg = onward(nr, nc);
31             if (best == -1 || deg < best) { best = deg; bestR = nr;
32                 bestC = nc; }
33         }
34         r = bestR; c = bestC;
35         board[r][c] = step;
36     }
37
38     for (int i = 0; i < 8; i++) {
39         for (int j = 0; j < 8; j++) cout << board[i][j] << " ";
40         cout << "\n";
41     }
42     return 0;
}

```

Complexity Analysis

Time Complexity. $O(64)$ moves, each evaluating a constant number of candidates – effectively constant for the fixed board. **Space Complexity.** $O(1)$ (a fixed 8×8 board).

Key Takeaway

Warnsdorff's “fewest onward moves” heuristic finds a knight's tour without backtracking by always escaping the most constrained square first. It is the practical alternative when a full Hamiltonian search would be too slow.

Chapter 7

Maximum Flow and Matching

Maximum flow is the master problem of this chapter. Once you can push the most units from a source to a sink through capacitated edges, you get minimum cuts, bipartite matchings, and edge-disjoint paths almost for free – each is a maximum flow with the graph set up just so.

7.1 Everything Is a Flow

A flow network has capacities on edges; the maximum flow from source s to sink t is the greatest total that can be pushed while respecting capacities and conservation. Augmenting-path algorithms (BFS-based Edmonds–Karp, or Dinic with level graphs) compute it.

Flow Reductions in This Chapter

- **Maximum throughput / edge-disjoint paths count – max flow** with unit or given capacities: Download Speed, Distinct Routes (unit capacities; then decompose the flow into paths).
- **Fewest edges to disconnect s from t – min cut** = max flow (max-flow–min-cut theorem), and the cut edges are those from reachable to unreachable after saturation: Police Chase.
- **Maximum bipartite matching – max flow** with a super-source into one side and a super-sink out of the other, all unit capacities: School Dance.

Residual Graphs and Augmenting Paths

Flow algorithms repeatedly find a path from s to t with spare capacity and push along it, adding *reverse* edges so earlier decisions can be undone. When no augmenting path remains, the flow is maximum, and the nodes still reachable from s define the minimum cut.

7.2 Download Speed

Problem Statement. Given a directed graph of n nodes and m edges, each with a capacity (bandwidth), find the maximum data rate from node 1 (source) to node n (sink).

Pattern Recognition

Maximum throughput under capacities is the **maximum flow** problem. **Dinic's algorithm** computes it efficiently: repeatedly build a level graph with BFS, then push blocking flow along shortest augmenting paths with DFS.

Level graphs and blocking flow

A flow assigns each edge a value up to its capacity, with flow conserved at every node except source and sink; the max flow is the greatest total leaving the source. Dinic alternates two phases: a BFS labels each node with its distance (level) from the source in the residual graph; a DFS then sends as much flow as possible along edges that go strictly one level deeper (the blocking flow). Reverse edges are maintained so earlier flow can be rerouted. When BFS can no longer reach the sink, the flow is maximum.

Augment Along Shortest Paths, in Bulk

Restricting augmenting paths to the level graph (only forward-level edges) and saturating them all before rebuilding gives Dinic its speed: the number of BFS phases is $O(V)$, far fewer than pushing one path at a time.

Algorithm

1. Build the flow network with forward capacities and zero-capacity reverse edges.
2. While BFS reaches the sink: DFS blocking flow along level edges, accumulating flow.
3. Output the total flow.

Dry run

Dry Run. $1 \rightarrow 2(3)$, $1 \rightarrow 3(2)$, $2 \rightarrow 4(2)$, $3 \rightarrow 4(3)$; sink 4.

- Push 2 along $1 \rightarrow 2 \rightarrow 4$ and 2 along $1 \rightarrow 3 \rightarrow 4$.
- Max flow = 4 (limited by outgoing/incoming capacities).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Dinic {
5     struct Edge { int to; long long cap; int rev; };
6     vector<vector<Edge>> g;
7     vector<int> level, it;
8     Dinic(int n) : g(n), level(n), it(n) {}
9     void addEdge(int u, int v, long long c) {
10         g[u].push_back({v, c, (int)g[v].size()});
11         g[v].push_back({u, 0, (int)g[u].size() - 1}); // reverse edge

```

```

12     }
13     bool bfs(int s, int t) {
14         fill(level.begin(), level.end(), -1);
15         queue<int> q; level[s] = 0; q.push(s);
16         while (!q.empty()) {
17             int u = q.front(); q.pop();
18             for (auto& e : g[u])
19                 if (e.cap > 0 && level[e.to] < 0) { level[e.to] = level
20                     [u] + 1; q.push(e.to); }
21         }
22         return level[t] >= 0;
23     }
24     long long dfs(int u, int t, long long f) {
25         if (u == t) return f;
26         for (int& i = it[u]; i < (int)g[u].size(); i++) {
27             Edge& e = g[u][i];
28             if (e.cap > 0 && level[u] < level[e.to]) {
29                 long long d = dfs(e.to, t, min(f, e.cap));
30                 if (d > 0) { e.cap -= d; g[e.to][e.rev].cap += d;
31                     return d; }
32             }
33         }
34         return 0;
35     }
36     long long maxflow(int s, int t) {
37         long long flow = 0;
38         while (bfs(s, t)) {
39             fill(it.begin(), it.end(), 0);
40             long long f;
41             while ((f = dfs(s, t, LLONG_MAX)) > 0) flow += f;
42         }
43         return flow;
44     }
45 };
46
47 int main() {
48     int n, m;
49     cin >> n >> m;
50     Dinic dinic(n + 1);
51     for (int i = 0; i < m; i++) {
52         int a, b; long long c; cin >> a >> b >> c;
53         dinic.addEdge(a, b, c);
54     }
55     cout << dinic.maxflow(1, n) << "\n";
56     return 0;
57 }

```

Complexity Analysis

Time Complexity. $O(V^2E)$ worst case (Dinic), far faster in practice. **Space Complexity.** $O(V + E)$.

Key Takeaway

Maximum throughput is maximum flow; Dinic's level-graph blocking flow is the standard efficient solver. This one algorithm is the engine behind the next three problems – min cut, matching, and edge-disjoint paths – each just a different network.

7.3 Police Chase

Problem Statement. Given an undirected graph of n intersections and m streets, find the minimum number of streets to remove so that node 1 is disconnected from node n , and output those streets.

Pattern Recognition

“Fewest edges to disconnect s from t ” is the **minimum cut**, which by the max-flow–min-cut theorem equals the **maximum flow** with every edge given capacity 1. The cut edges are those crossing from the source-reachable side to the rest after the flow saturates.

Max flow, then read the cut

Model each street as an undirected unit-capacity edge (a pair of directed edges). The max flow from 1 to n equals the minimum number of streets whose removal disconnects them. After computing it, BFS in the residual graph from node 1: the reachable set S is one side of the cut. Every original street with exactly one endpoint in S is a cut edge – print those. Their count equals the max-flow value.

The Reachable Side Defines the Cut

When no augmenting path remains, the nodes still reachable from the source in the residual graph are separated from the rest by saturated edges. The original edges straddling that boundary are precisely a minimum edge cut.

Algorithm

1. Add each street as a unit-capacity edge in both directions; run max flow $1 \rightarrow n$.
2. BFS the residual graph from 1 to get the reachable set S .
3. Output every original street with one endpoint in S and one outside.

Dry run

Dry Run. Path 1–2–3 with $n = 3$; a single street between 2 and 3 (and 1, 2).

- Max flow = 1; residual-reachable set from 1 is $\{1, 2\}$.
- The street 2–3 crosses the boundary – remove it (one street).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Dinic {
5     struct Edge { int to; long long cap; int rev; };
6     vector<vector<Edge>> g;
7     vector<int> level, it;
8     Dinic(int n) : g(n), level(n), it(n) {}
9     void addEdge(int u, int v, long long c) {
10         g[u].push_back({v, c, (int)g[v].size()});
11         g[v].push_back({u, c, (int)g[u].size() - 1});    // undirected:
12                     both cap c

```

```

12     }
13     bool bfs(int s, int t) {
14         fill(level.begin(), level.end(), -1);
15         queue<int> q; level[s] = 0; q.push(s);
16         while (!q.empty()) {
17             int u = q.front(); q.pop();
18             for (auto& e : g[u])
19                 if (e.cap > 0 && level[e.to] < 0) { level[e.to] = level
20                     [u] + 1; q.push(e.to); }
21         }
22         return level[t] >= 0;
23     }
24     long long dfs(int u, int t, long long f) {
25         if (u == t) return f;
26         for (int& i = it[u]; i < (int)g[u].size(); i++) {
27             Edge& e = g[u][i];
28             if (e.cap > 0 && level[u] < level[e.to]) {
29                 long long d = dfs(e.to, t, min(f, e.cap));
30                 if (d > 0) { e.cap -= d; g[e.to][e.rev].cap += d;
31                     return d; }
32             }
33         }
34         return 0;
35     }
36     long long maxflow(int s, int t) {
37         long long flow = 0;
38         while (bfs(s, t)) {
39             fill(it.begin(), it.end(), 0);
40             long long f;
41             while ((f = dfs(s, t, LLONG_MAX)) > 0) flow += f;
42         }
43         return flow;
44     }
45 };
46
47 int main() {
48     int n, m;
49     cin >> n >> m;
50     Dinic dinic(n + 1);
51     vector<pair<int, int>> streets(m);
52     for (auto& [a, b] : streets) { cin >> a >> b; dinic.addEdge(a, b,
53         1); }
54
55     long long cut = dinic.maxflow(1, n);
56     // residual reachability from source
57     vector<bool> reach(n + 1, false);
58     queue<int> q; q.push(1); reach[1] = true;
59     while (!q.empty()) {
60         int u = q.front(); q.pop();
61         for (auto& e : dinic.g[u])
62             if (e.cap > 0 && !reach[e.to]) { reach[e.to] = true; q.push
63                 (e.to); }
64     }
65
66     cout << cut << "\n";
67     for (auto& [a, b] : streets)
68         if (reach[a] != reach[b]) cout << a << " " << b << "\n";
69     return 0;

```

66 }

Complexity Analysis

Time Complexity. $O(V^2E)$ for the flow; the cut extraction is $O(V + E)$. **Space Complexity.** $O(V + E)$.

Key Takeaway

Minimum edge cut equals maximum flow with unit capacities, and the cut edges straddle the residual-reachable frontier. Max-flow–min-cut is one of the most powerful reductions in graph theory – “disconnect cheaply” becomes “push maximally.”

7.4 School Dance

Problem Statement. There are n boys and m girls, and k acceptable boy–girl pairs. Form the maximum number of dancing pairs so that each person is in at most one pair, and output the pairs.

Pattern Recognition

Maximum **bipartite matching** is a **maximum flow**: add a super- source into every boy, edges for each acceptable pair, and every girl into a super-sink, all with capacity 1. The max flow equals the maximum matching, and the used pair-edges are the matching.

Matching as unit-capacity flow

Build nodes: source S , boys $1 \dots n$, girls $1 \dots m$, sink T . Add $S \rightarrow \text{boy}$ (cap 1, so each boy is used at most once), $\text{boy} \rightarrow \text{girl}$ for each acceptable pair (cap 1), and $\text{girl} \rightarrow T$ (cap 1). A unit of flow from S to T traces a boy–girl pairing, and capacity 1 everywhere enforces “at most one partner.” The max flow is the largest matching; a boy–girl edge that ended up saturated is a matched pair.

Unit Capacities Enforce “At Most One”

Capacity-1 edges out of the source (per boy) and into the sink (per girl) guarantee each person is matched once. Integral max flow then corresponds exactly to a maximum matching – the classic flow reduction.

Algorithm

1. Build the source/boys/girls/sink network with unit capacities.
2. Run max flow; its value is the matching size.
3. For each pair-edge $\text{boy} \rightarrow \text{girl}$ whose forward capacity is now 0, output the pair.

Dry run

Dry Run. Boys $\{1, 2\}$, girls $\{1, 2\}$; pairs $(1, 1), (1, 2), (2, 1)$.

- Match boy 1–girl 2 and boy 2–girl 1: two pairs.
- Max flow = 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Dinic {
5     struct Edge { int to; long long cap; int rev; };
6     vector<vector<Edge>> g;
7     vector<int> level, it;
8     Dinic(int n) : g(n), level(n), it(n) {}
9     int addEdge(int u, int v, long long c) {
10         g[u].push_back({v, c, (int)g[v].size()});
11         g[v].push_back({u, 0, (int)g[u].size() - 1});
12         return (int)g[u].size() - 1; // index of
            forward edge
13     }
14     bool bfs(int s, int t) {
15         fill(level.begin(), level.end(), -1);
16         queue<int> q; level[s] = 0; q.push(s);
17         while (!q.empty()) {
18             int u = q.front(); q.pop();
19             for (auto& e : g[u])
20                 if (e.cap > 0 && level[e.to] < 0) { level[e.to] = level
                    [u] + 1; q.push(e.to); }
21         }
22         return level[t] >= 0;
23     }
24     long long dfs(int u, int t, long long f) {
25         if (u == t) return f;
26         for (int& i = it[u]; i < (int)g[u].size(); i++) {
27             Edge& e = g[u][i];
28             if (e.cap > 0 && level[u] < level[e.to]) {
29                 long long d = dfs(e.to, t, min(f, e.cap));
30                 if (d > 0) { e.cap -= d; g[e.to][e.rev].cap += d;
                    return d; }
31             }
32         }
33         return 0;
34     }
35     long long maxflow(int s, int t) {
36         long long flow = 0;
37         while (bfs(s, t)) {
38             fill(it.begin(), it.end(), 0);
39             long long f;
40             while ((f = dfs(s, t, LLONG_MAX)) > 0) flow += f;
41         }
42         return flow;
43     }
44 };
45
46 int main() {
47     int n, m, k;
48     cin >> n >> m >> k;
49     int S = 0, T = n + m + 1; // boys 1..n,
        girls n+1..n+m
50     Dinic dinic(T + 1);
51     for (int i = 1; i <= n; i++) dinic.addEdge(S, i, 1);
52     for (int j = 1; j <= m; j++) dinic.addEdge(n + j, T, 1);

```

```

53     vector<array<int,3>> pairEdge;                                // {boy, girl,
54         forward edge index}
55     for (int i = 0; i < k; i++) {
56         int a, b; cin >> a >> b;                                // boy a, girl b
57         int idx = dinic.addEdge(a, n + b, 1);
58         pairEdge.push_back({a, n + b, idx});
59     }
60
61     cout << dinic.maxflow(S, T) << "\n";
62     for (auto& [boy, girlNode, idx] : pairEdge)
63         if (dinic.g[boy][idx].cap == 0)                          // saturated =>
64             cout << boy << " " << girlNode - n << "\n";
65     return 0;
66 }
```

Complexity Analysis

Time Complexity. $O(E\sqrt{V})$ for bipartite matching via Dinic. **Space Complexity.** $O(V + E)$.

Key Takeaway

Bipartite matching is unit-capacity max flow with a super-source and super-sink; saturated pair-edges are the matches. This reduction makes matching, assignment, and covering problems all instances of flow.

7.5 Distinct Routes

Problem Statement. Given a directed graph of n nodes and m edges, find the maximum number of *edge-disjoint* routes from node 1 to node n , and output each route. Every edge may be used by at most one route.

Pattern Recognition

The maximum number of edge-disjoint paths from s to t equals the **maximum flow** when every edge has capacity 1 (Menger's theorem). After computing the flow, **decompose** it into that many paths by following edges that carry flow.

Max flow, then decompose into paths

Give every edge capacity 1 and compute the max flow from 1 to n ; its value f is the number of edge-disjoint routes. An original edge $u \rightarrow v$ carries flow iff its forward capacity dropped to 0 (equivalently its reverse now has capacity 1). To extract the routes, walk from the source: at each node follow one unused flow-carrying out-edge, mark it consumed, and continue to the sink; repeat f times. Because the flow is edge-disjoint, these walks partition the flow edges into f paths.

Flow Decomposition Yields the Paths

A unit s - t flow of value f always splits into f edge-disjoint $s \rightarrow t$ paths (plus possibly some cycles, which unit capacities here avoid). Greedily tracing flow edges from the source recovers each path exactly once.

Algorithm

1. Add each edge with capacity 1; run max flow $1 \rightarrow n$; let f be its value.
2. Collect, per node, the out-edges that carry flow.
3. Trace f paths from 1 to n , consuming each flow edge once; print the count and paths.

Dry run

Dry Run. $1 \rightarrow 2$, $2 \rightarrow 4$, $1 \rightarrow 3$, $3 \rightarrow 4$; $n = 4$.

- Max flow = 2: routes $1 \rightarrow 2 \rightarrow 4$ and $1 \rightarrow 3 \rightarrow 4$.
- Both routes are edge-disjoint.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Dinic {
5     struct Edge { int to; long long cap; int rev; };
6     vector<vector<Edge>> g;
7     vector<int> level, it;
8     Dinic(int n) : g(n), level(n), it(n) {}
9     void addEdge(int u, int v, long long c) {
10         g[u].push_back({v, c, (int)g[v].size()});
11         g[v].push_back({u, 0, (int)g[u].size() - 1});
12     }
13     bool bfs(int s, int t) {
14         fill(level.begin(), level.end(), -1);
15         queue<int> q; level[s] = 0; q.push(s);
16         while (!q.empty()) {
17             int u = q.front(); q.pop();
18             for (auto& e : g[u])
19                 if (e.cap > 0 && level[e.to] < 0) { level[e.to] = level
20                     [u] + 1; q.push(e.to); }
21         }
22         return level[t] >= 0;
23     }
24     long long dfs(int u, int t, long long f) {
25         if (u == t) return f;
26         for (int& i = it[u]; i < (int)g[u].size(); i++) {
27             Edge& e = g[u][i];
28             if (e.cap > 0 && level[u] < level[e.to]) {
29                 long long d = dfs(e.to, t, min(f, e.cap));
30                 if (d > 0) { e.cap -= d; g[e.to][e.rev].cap += d;
31                     return d; }
32             }
33         }
34         return 0;
35     }
36     long long maxflow(int s, int t) {
37         long long flow = 0;
38         while (bfs(s, t)) {
39             fill(it.begin(), it.end(), 0);
40             long long f;

```

```

39         while ((f = dfs(s, t, LLONG_MAX)) > 0) flow += f;
40     }
41     return flow;
42 }
43 };
44
45 int main() {
46     int n, m;
47     cin >> n >> m;
48     Dinic dinic(n + 1);
49     for (int i = 0; i < m; i++) {
50         int a, b; cin >> a >> b;
51         dinic.addEdge(a, b, 1);
52     }
53
54     long long f = dinic.maxflow(1, n);
55     cout << f << "\n";
56
57     // flow-carrying out-edges: forward edge with cap now 0 (capacity
58     // was 1)
59     vector<int> ptr(n + 1, 0);
60     for (long long r = 0; r < f; r++) {
61         vector<int> path;
62         int u = 1;
63         path.push_back(1);
64         while (u != n) {
65             auto& es = dinic.g[u];
66             while (ptr[u] < (int)es.size() &&
67                    !(es[ptr[u]].to != 0 && es[ptr[u]].cap == 0 &&
68                      dinic.g[es[ptr[u]].to][es[ptr[u]].rev].cap > 0))
69                 ptr[u]++;
70             auto& e = es[ptr[u]];
71             dinic.g[e.to][e.rev].cap = 0;           // consume this
72             // flow edge
73             u = e.to;
74             path.push_back(u);
75         }
76         cout << path.size() << "\n";
77         for (int v : path) cout << v << " ";
78         cout << "\n";
79     }
80     return 0;
81 }

```

Complexity Analysis

Time Complexity. $O(V^2E)$ for the flow; decomposition is $O(V + E)$ across all paths.
Space Complexity. $O(V + E)$.

Key Takeaway

The most edge-disjoint s - t paths is unit-capacity max flow (Menger's theorem), and flow decomposition recovers the actual routes. This closes the flow chapter: throughput, cut, matching, and disjoint paths are one algorithm applied to four networks.